

Numerical integration of highly oscillatory functions

Morten M. Jensen (morje21@student.sdu.dk)
Ole W. Kaa (olkaa21@student.sdu.dk)

Bachelor Thesis
IMADA

Supervisor: Benjamin Jäger



Abstract

I denne bachelorafhandling studeres problemet med at integrere højt oscillerende funktioner med numeriske metoder. Højt oscillerende funktioner er ofte svære at integrere numerisk på grund af fortegnsproblemet, som er, når de positive og negative dele af integralet er næsten ens. De udligner derfor hinanden, når de bliver numerisk approksimeret. Det gør at numeriske integrationsmetoder ofte afviger stort fra det sande integral. I denne afhandling bliver en specifik højt oscillerende funktion med et kendt integral brugt til at undersøge, hvor akkurate forskellige integrationsmetoder er.

Til at starte med bliver forskellige integrationsmetoder præsenteret og diskuteret. Disse metoder er Newton-Cotes-metoder i én dimension og Monte Carlo integrationsmetoder i både én og flere dimensioner. Køretiden og den numeriske præcision for disse metoder bliver også diskuteret. Bagefter bliver integralet af funktionen på et særligt interval udledt analytisk i både én og flere dimensioner. På den måde fås et resultat, som kan bruges til at vurdere præcisionen af de numeriske integrationsmetoder. Bagefter udledes Newton-Cotes-metoder for todimensionelle funktioner. Disse metoder skal bruges til at approksimere integralet af den todimensionelle version af funktionen. Der bliver også udledt en formel for, hvordan afvigelsen fra den sande løsning falder med flere punkter, og hvad køretiden er for disse metoder.

Når alt det nødvendige baggrundsmateriale er diskuteret, bliver der udført numeriske integration for at vurdere præcisionen af de diskuteret metoder. Præcisionen bliver studeret for forskellige antal af skridt for Newton-Cotes-metoderne og antal punkter for Monte Carlo integrationsmetoderne. Præcisionen bliver også vurderet på forskellige værdier, der styrer, hvor meget funktionen oscillerer. Der bliver præsenteret grafer og figurer, som viser, hvordan afvigelsen fra det sande integral aftager for de forskellige metoder, når antallet af skridt/punkter vokser. Der bliver også præsenteret grafer, som viser forskellen på, hvordan afvigelsen aftager for Newton-Cotes-metoderne sammenlignet med Monte Carlo integrationsmetoderne. Der bliver også undersøgt, hvordan dimensionen af funktionen påvirker køretiden og præcision af de forskellige metoder.

Herefter bliver der konkluderet, at Newton-Cotes-metoder generelt har en lavere afvigelse fra det sande integral end Monte Carlo integrationsmetoder. Den højere præcision er ikke mere krævende i forhold til køretiden for funktioner i lavere dimensioner. Efter det bliver der konkluderet, at funktionens oscillerende adfærd generelt forøger afvigelsen for alle metoder, men ikke påvirker hvordan afvigelsen aftager, når antallet af punkter vokser. Til sidst bliver der konkluderet, at Monte Carlo integrationsmetoder forventes at have lavere køretid end Newton-Cotes-metoder for funktioner i højere dimensioner.

Contents

1	Introduction	2
2	Background	2
2.1	Oscillating function	2
2.2	Generalized Intermediate Value Theorem	4
2.3	Newton-Cotes methods in one dimension	5
2.3.1	Midpoint rule	5
2.3.2	Trapezoid rule	5
2.3.3	Simpson's rule	6
2.4	Monte Carlo integration	7
2.4.1	Simple sampling	7
2.4.2	Importance sampling	7
2.4.3	Computational complexity of Monte Carlo integration	8
2.4.4	Variance of Monte Carlo integration	8
3	True solution of integral	9
3.1	True solution of one-dimensional integral	9
3.2	True solution of one-dimensional bounded integral	10
3.3	True solution of multi-dimensional integral	11
3.3.1	Base case: $d = 1$	12
3.3.2	Inductive hypothesis	12
3.3.3	Induction step	13
3.4	True solution of multi-dimensional bounded integral	14
4	Extending Newton-Cotes methods to two dimensions	14
4.1	Composite midpoint rule	14
4.1.1	Error of 2D composite midpoint rule	15
4.2	Composite trapezoid rule	17
4.2.1	Error of 2D composite trapezoid rule	18
4.3	Composite Simpson's rule	21
4.3.1	Error of 2D composite Simpson's rule	23
4.4	Computational complexity of higher dimensional Newton-Cotes methods	26
5	Numerical experiments	26
5.1	One-dimensional Newton-Cotes methods	26
5.2	One-dimensional Monte Carlo integration methods	28
5.3	Error comparison between Newton-Cotes and Monte Carlo methods in one dimension	31
5.4	Two-dimensional Newton-Cotes methods	32
5.5	Two-dimensional Monte Carlo integration methods	33
5.6	Error comparison between Newton-Cotes and Monte Carlo methods in two dimensions	35
6	Conclusion	37
7	Bibliography	37
8	Appendix	37

1 Introduction

In the study of functions and their behavior, it is often necessary to integrate the function. Some functions can be integrated analytically and the value of their integral can be computed. However, there are other functions that are either difficult or impossible to integrate analytically, so one may need to numerically approximate the integral. This can be done with numerical integration methods. Here it is expected that the approximations are not exactly equal to the true solution of the integral. How close the approximation is compared to the true solution varies depending on the method of integration that is being used. The accuracy will also depend on the behavior of the function itself.

One example where the behavior of the function has a large effect on the numerical accuracy of the approximation is when the function is highly oscillatory. These kinds of functions have what is called the sign problem, which is when the positive and negative contributions of the integral are almost equal to each other. They therefore often cancel each other out when approximated numerically.

An example of such a function is $f(x) = e^{-x^2} \cos(\lambda x)$, $\lambda \in \mathbb{R}$. Unlike some other functions that have the sign problem, this function can be integrated analytically. It is therefore possible to assess the accuracy of numerical integration methods by comparing their results with the true solution of the integral of this function.

In addition to one-dimensional functions like this one, higher dimensional functions can also have the sign problem. Therefore, in addition to the numerical accuracy of the given method, one can also study the computational complexity of a given integration method and how it varies depending on the dimension and on other parameters.

In this thesis, we will be studying how different numerical integration methods work when evaluating the integral of the given function for different values of λ . We will study how the error varies depending on the method, how the error can be reduced, and how the oscillating behavior of the function affects the error.

2 Background

Before we numerically integrate the function, we need some results and facts about different topics which will be used later. We begin by talking about the function we want to integrate and its behavior.

2.1 Oscillating function

For this project we look at a particular function:

$$f(x) = e^{-x^2} \cos(\lambda x), \lambda \in \mathbb{R}$$

If the value of $\lambda = 0$, then the function is just the Gaussian exponential function e^{-x^2} . If $\lambda \neq 0$, then the function oscillates like a cosine function. The function is even, [1], which we will use in numerical experiments. Graphs of the function plotted on the interval $[-4, 4]$ for four different values of $\lambda \in \{1, 3, 6, 8\}$ can be seen in figure 1. For the numerical integration, we will integrate this function on the interval $[-3, 3]$ and with these values of λ . The reason we use this particular interval is because that is where the function has the largest values and at the values outside of the interval, the function values become small because of the Gaussian exponential function.

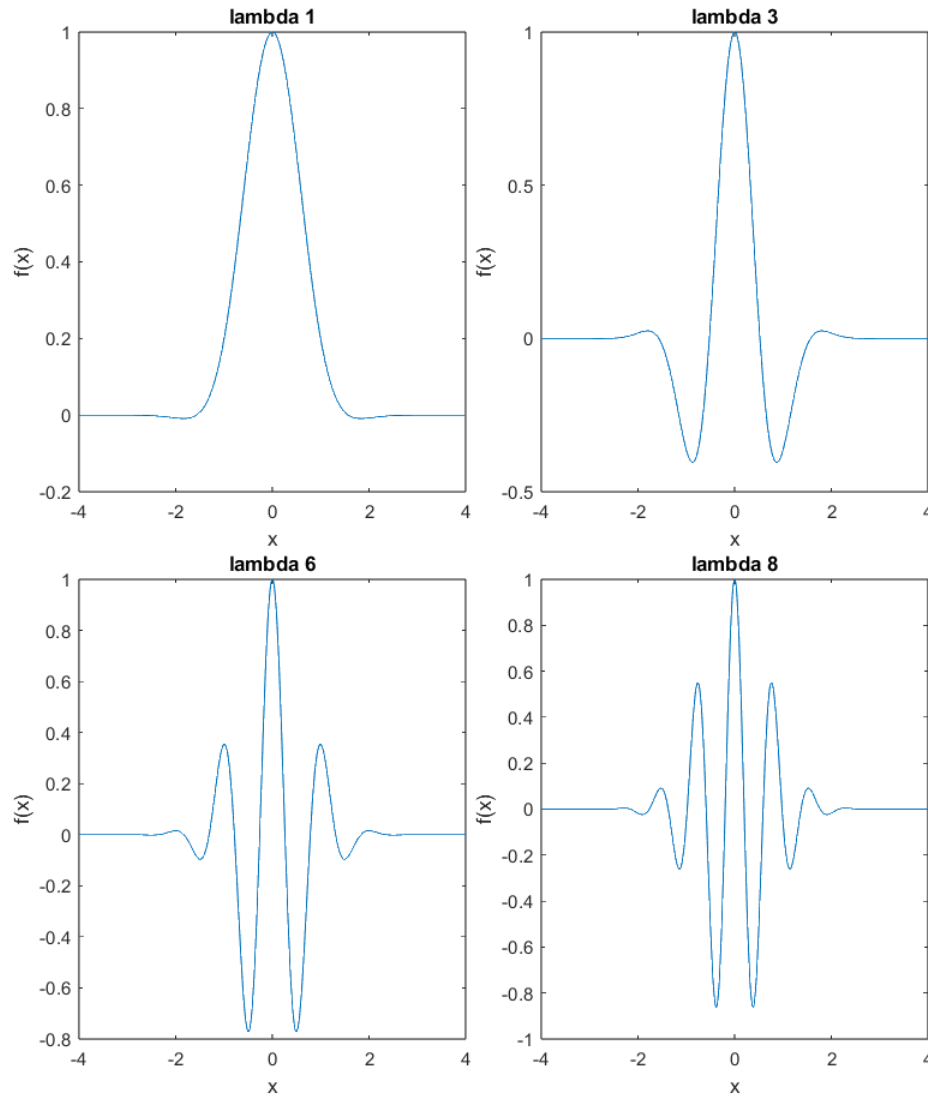


Figure 1: The function $e^{-x^2} \cos(\lambda x)$ plotted on the interval $[-4, 4]$ for $\lambda \in \{1, 3, 6, 8\}$. Made with `funcPlot.m`.

The function can also be generalised to a d -dimensional function:

$$f(x_1, x_2, \dots, x_d) = e^{-\sum_{i=1}^d x_i^2} \cos\left(\sum_{i=1}^d \lambda_i x_i\right), \quad \lambda_i \in \mathbb{R}, \quad \forall i \in \{1, \dots, d\}$$

If all values of λ are equal to 0, then the function is the multi-dimensional Gaussian exponential function. It begins to oscillate when the values of the $\lambda_i \neq 0$. A surface plot for the two-dimensional function on the domain $[-3, 3]^2$, with $\lambda_1, \lambda_2 \in \{1, 3, 6, 8\}$ and $\lambda_1 = \lambda_2$, is shown in figure 2. For the numerical integration, we will integrate the function for these values of λ on the domain $[-3, 3]^2$.

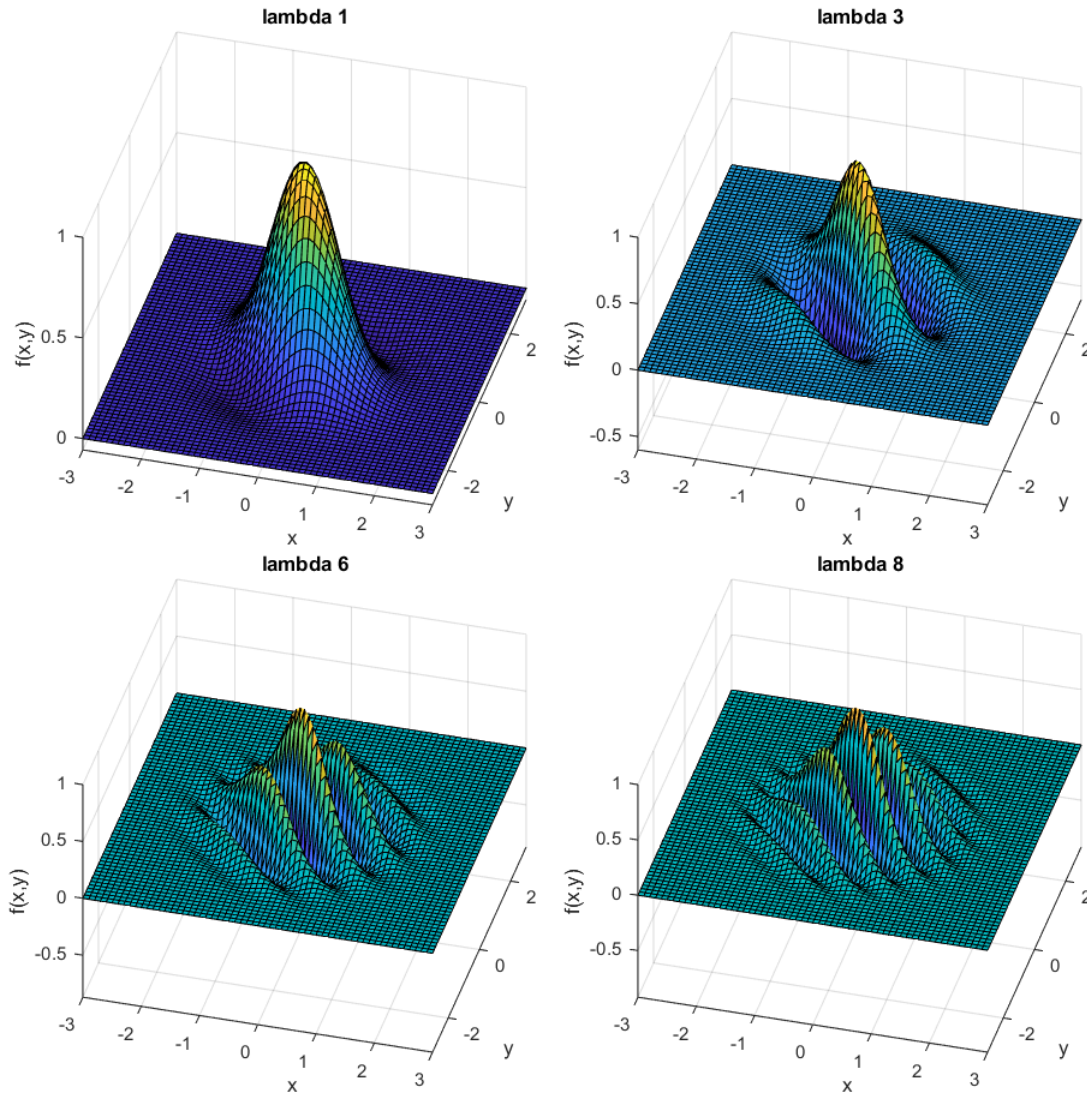


Figure 2: The function $e^{-(x_1^2+x_2^2)} \cos(\lambda_1 x_1 + \lambda_2 x_2)$ plotted on the domain $[-3, 3]^2$ for $\lambda_1, \lambda_2 \in \{1, 3, 6, 8\}$ and $\lambda_1 = \lambda_2$. Made with `surfPlots.m`.

We will now discuss a result that we will use later when deriving the error terms for higher dimensional numerical integration methods.

2.2 Generalized Intermediate Value Theorem

Before we discuss each of the Newton-Cotes methods to be used later, we will mention another result, the Generalized Intermediate Value Theorem. Let f be a continuous function on a interval $[a, b]$. Then let $x_1, \dots, x_n$ be points in $[a, b]$ and $a_1, \dots, a_n > 0$. Then there exists a number c between a and b such that, [5, p. 245]:

$$(a_1 + \dots + a_n)f(c) = a_1 f(x_1) + \dots + a_n f(x_n) \quad (2.2.1)$$

This results will be useful when we derive the error-terms for the 2-dimensional Newton-Cotes methods.

2.3 Newton-Cotes methods in one dimension

One of the ways to numerically approximate the integral of a function over a bounded domain is Newton-Cotes methods. In particular, we will be using the midpoint rule, the trapezoid rule and the Simpson's rule. The general idea of Newton-Cotes methods is to numerically approximate the integral of a function $f(x)$ over an interval $[a, b]$ using a sum of function values evaluated at different points on the interval. How this is done depends on the method used.

2.3.1 Midpoint rule

The midpoint rule is an example of an open Newton-Cotes method. This means that the method does not use the function values at the points a and b when approximating the integral. Instead, the midpoint rule partitions the interval $[a, b]$ into n subintervals $[x_0, x_1], [x_1, x_2], \dots, [x_{n-2}, x_{n-1}], [x_{n-1}, x_n]$, with step size $h = \frac{(b-a)}{n}$. The integral of a function $f(x)$ over a subinterval $[x_{i-1}, x_i]$ is approximated as follows, [5, p. 262]:

$$\int_{x_{i-1}}^{x_i} f(x) dx \approx hf(w_i), \quad w_i = x_{i-1} + \frac{h}{2}, \quad i = 1, \dots, n$$

Adding up the terms for the integral over each subinterval, the composite midpoint rule can be written as follows, [5, p. 263]:

$$\int_a^b f(x) dx \approx h \sum_{i=1}^n f(w_i)$$

The error term is $\frac{(b-a)h^2}{24} f''(c)$ for some $c \in [a, b]$, [5, p. 263]. The rate of decay of the error of the numerical approximation of the integral is therefore $\mathcal{O}(h^2)$, so the composite midpoint rule is accurate of order 2. For comparison with other types of numerical integration methods, it is better to have the order of the error based on the number of steps, where the number of steps is equal to the number of subintervals. Because the step size and number of steps are inversely proportional, $h = \frac{(b-a)}{n}$. It can be substituted into the error term:

$$\frac{(b-a)h^2}{24} f''(c) = \frac{(b-a)^3}{24n^2} f''(c)$$

Meaning the rate of decay of the error is $\mathcal{O}(n^{-2})$.

We also need the computational complexity of the different Newton-Cotes methods. The computational complexity for each method is proportional to how many times the function we integrate needs to be evaluated.

For the midpoint rule, we have a sum of n terms that involve the function. The computational complexity is therefore $\mathcal{O}(n)$.

2.3.2 Trapezoid rule

Trapezoid rule and Simpson's rule are examples of closed Newton-Cotes methods meaning that they use the function values at the boundaries of the interval. The setting for the trapezoid rule is the same as the midpoint rule, that is we partition the interval $[a, b]$ into n subintervals of the form $[x_{i-1}, x_i]$, $i = 1, \dots, n$ and with step size $h = \frac{(b-a)}{n}$. We then consider the approximation of the integral over each interval. The formula for the trapezoid rule for each subinterval is, [5, p. 257]:

$$\int_{x_{i-1}}^{x_i} f(x) dx \approx \frac{h}{2} (f(x_{i-1}) + f(x_i)), \quad i = 1, \dots, n$$

The composite trapezoid rule over the interval $[a, b]$ can then be written as, [5, p. 260]:

$$\int_a^b f(x) dx \approx \frac{h}{2} \left(f(x_0) + f(x_n) + 2 \sum_{i=1}^{n-1} f(x_i) \right)$$

The function values at the boundaries only appear once each, while every other term in the sum is added twice. That is why the sum over function values not evaluated at the ends of the interval are multiplied by 2. The error term for the trapezoid rule is $-\frac{(b-a)h^2}{12}f''(c)$ for some $c \in [a, b]$, [5, p. 260]. The trapezoid rule is therefore a order 2 method like the midpoint rule. Again we want the order of the error based on the number of steps, substituting $h = \frac{(b-a)}{n}$ into the error terms:

$$-\frac{(b-a)h^2}{12}f''(c) = -\frac{(b-a)^3}{12n^2}f''(c)$$

Meaning the rate of decay of the error is $\mathcal{O}(n^{-2})$.

Similar to the midpoint rule, we need the computational complexity of the trapezoid rule. The function is evaluated $n + 1$ times which means the computational complexity is of order $\mathcal{O}(n)$.

2.3.3 Simpson's rule

As mentioned earlier, Simpson's rule is also a closed Newton-Cotes method that includes the function values at the boundaries of the interval. The difference is that the Simpson's rule uses more terms in the integral approximation. We still let n be some integer, but now the interval $[a, b]$ is partitioned into $2n$ subintervals $[x_0, x_1], [x_1, x_2], \dots, [x_{2n-1}, x_{2n}]$. Instead of approximating the integral over $[x_{i-1}, x_i]$, we now approximate the integral over the subintervals $[x_{i-2}, x_i]$, $i \in \{2, 4, \dots, 2(n-1), 2n\}$. This means that the step size is $h = \frac{(b-a)}{2n}$. The numerical approximation of the integral over one such subinterval using Simpson's rule is, [5, p. 257]:

$$\int_{x_{i-2}}^{x_i} f(x) dx = \frac{h}{3} (f(x_{i-2}) + 4f(x_{i-1}) + f(x_i)) \quad i \in \{2, 4, \dots, 2(n-1), 2n\}$$

The composite Simpson's rule becomes, [5, p. 261]:

$$\int_a^b f(x) dx \approx \frac{h}{3} \left(f(x_0) + f(x_{2n}) + 4 \sum_{i=1}^n f(x_{2i-1}) + 2 \sum_{i=1}^{n-1} f(x_{2i}) \right)$$

The function values evaluated at the boundaries of the interval appear once each, the function values evaluated at indices $2i$, $i = 1, \dots, n-1$ each appear twice, once in the approximation of the integral over the interval $[x_{i-2}, x_i]$ and once in $[x_i, x_{i+2}]$, and the terms indexed $2i-1$, $i = 1, \dots, n$ each appear once. The error term for Simpson's rule is $-\frac{(b-a)h^4}{180}f^{(iv)}(c)$ for some $c \in [a, b]$, [5, p. 261]. Simpson's rule is therefore accurate of order $\mathcal{O}(h^4)$. For comparison with other types of integration methods, we substitute $h = \frac{(b-a)}{2n}$ into the error term:

$$-\frac{(b-a)h^4}{180}f^{(iv)}(c) = -\frac{(b-a)^5}{2880n^4}f^{(iv)}(c)$$

Meaning the order of the Simpson's rule is $\mathcal{O}(n^{-4})$.

As for the computational complexity, the Simpson's rule evaluates the function $2n + 1$ times, so the computational complexity is proportional to $\mathcal{O}(n)$.

2.4 Monte Carlo integration

Another way to numerically approximate the integral of a function is with Monte Carlo integration. We will present both simple sampling and importance sampling.

2.4.1 Simple sampling

Simple sampling is a way of numerically approximating the integral of a function on a closed interval. We want to approximate the integral:

$$\int_a^b f(x) dx$$

For some function $f(x)$. Simple sampling samples from the uniform distribution on the interval $[a, b]$, to approximate the integral as follows, [4, p. 2]:

$$\int_a^b f(x) dx \approx \frac{(b-a)}{N} \sum_{i=1}^N f(x_i), \quad x_i \sim U(a, b)$$

Where N is the sample size. The sum is normalised by the length of the interval $(b-a)$. Simple sampling can also be generalised to the integral of a n -dimensional function over a n -dimensional volume V :

$$I = \int_V f(\mathbf{x}) dV$$

Where $f(\mathbf{x})$ is a function of a n -dimensional vector $\mathbf{x}$. The integral can be approximated by:

$$I \approx \frac{V}{N} \sum_{i=1}^N f(\mathbf{x}_i) \quad \mathbf{x}_i \sim U_n(V)$$

2.4.2 Importance sampling

Another form of Monte Carlo integration is importance sampling. Importance sampling is similar to simple sampling in that it involves sampling random points and evaluating the mean of function values at those points. In fact, simple sampling can be written as a special case of importance sampling. Importance sampling differs in that instead of sampling the uniform distribution, it instead samples points according to some other distribution.

We consider some weight function $w(x)$ which may or may not be normalised, that is to say $\int w(x) dx = 1$. If $w(x)$ is not normalised, we can instead define $p(x) = \frac{w(x)}{\int w(x) dx}$, so $p(x)$ is normalised, [4, p. 4]. We can approximate the integral of $f(x)$ by sampling N points and estimating the integral like this:

$$I \approx \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{p(x_i)} = \frac{\int w(x) dx}{N} \sum_{i=1}^N \frac{f(x_i)}{w(x_i)}$$

Simple sampling is importance sampling, where the weight function is the pdf of $U(a, b)$ random variable. Then the probability density function $w(x) = \frac{1}{b-a}$ which is normalised. Importance sampling is used to get a more reliable estimate of the integral of f using fewer points by changing the function ratio $\frac{f(x)}{w(x)}$ into something that can be sampled with fewer points. The error of the integral approximation using importance sampling is now dependent on the variance of the function ratio.

2.4.3 Computational complexity of Monte Carlo integration

When simulating, the computational complexity of Monte Carlo depends on the sample size N . When integrating a higher dimensional function, each of the N sampled points will be a d -dimensional vector of points that need to be generated. This means the computational complexity for Monte Carlo integration on a d -dimensional function is $\mathcal{O}(dN)$.

2.4.4 Variance of Monte Carlo integration

In order to assess the numerical accuracy of Monte Carlo integration, we need the rate at which the variance of the integral estimator decays as $N \rightarrow \infty$. We will therefore derive the standard deviation of the simple sampling estimator. We consider the random variable $Y = f(X)$, where X is the random variable that is used for sampling in Monte Carlo integration. We let $E(Y) = \mu$ and $\text{Var}(Y) = \sigma^2$. We then look at the sample mean for this random variable:

$$\bar{Y} = \frac{1}{N} \sum_{i=1}^N Y_i, \quad Y_i \sim Y$$

If we take the variance of the sample mean, we can derive the following:

$$\text{Var}(\bar{Y}) = \frac{1}{N^2} \text{Var}\left(\sum_{i=1}^N Y_i\right)$$

Since the Y_i 's are equal in distribution, they all have the same variance. And since they are independently generated, we can split the variance into a sum of variances:

$$= \frac{1}{N^2} \sum_{i=1}^N \text{Var}(Y_i) = \frac{N}{N^2} \text{Var}(Y) = \frac{\text{Var}(Y)}{N}$$

We can take the square root of this expression to find the standard deviation, which is the expected error:

$$\sqrt{\frac{\text{Var}(Y)}{N}} = \frac{\sigma}{\sqrt{N}}$$

We can now see that decay of the standard deviation of the Monte Carlo integration estimator of the integral is of order $\mathcal{O}(N^{-\frac{1}{2}})$.

For importance sampling the random variable Y is defined as $Y = \frac{f(X)}{w(X)}$. From there, the same assumptions as with simple sampling leads to the same rate of decay for the standard deviation of the estimator. The difference is the value of σ , so the rate of decay is still of order $\mathcal{O}(N^{-\frac{1}{2}})$.

In higher dimensions we consider $Y = f(\mathbf{X})$, where $\mathbf{X}$ is a multivariate random variable and f is a multi-dimensional function. Y is therefore still a one-dimensional random variable and the rate of decay of the error is $\mathcal{O}(N^{-\frac{1}{2}})$ for the same reasons as the one-dimensional case.

Now we have the necessary background knowledge for the rest of the thesis. In the following section we will analytically integrate our function for different bounds and dimensions, so we have a value to compare the results from the numerical integration methods to. After that, we will derive the two-dimensional Newton-Cotes methods.

3 True solution of integral

We are interested in numerically approximating the true integrals of highly oscillatory functions. We will therefore look at one such function where the true solution is known and can be derived. The function is $e^{-x^2} \cos(\lambda x)$ for some scalar λ and the integral of this function is known:

$$\int_{-\infty}^{\infty} e^{-x^2} \cos(\lambda x) dx = \sqrt{\pi} e^{-\frac{\lambda^2}{4}}$$

In this section we will show how integrate the function from $-\infty$ to ∞ . We will also show the true integral when we instead restrict the domain of integration to a symmetric bound $[-B, B]$, $B \in \mathbb{R}$. We will also show how to integrate the multi-dimensional function $e^{-\sum_{i=1}^n x_i^2} \cos(\sum_{i=1}^n \lambda_i x_i)$ on both the entire domain $\mathbb{R}^n$ and on a subdomain $[-B_1, B_1] \times [-B_2, B_2] \times \dots \times [-B_n, B_n]$, $(B_1, B_2, \dots, B_n)^T \in \mathbb{R}^n$.

3.1 True solution of one-dimensional integral

First we will show how to integrate the one-dimensional version of the function over the real line. We consider the integral:

$$\int_{-\infty}^{\infty} e^{-x^2} \cos(\lambda x) dx \quad (3.1.1)$$

The first thing we can do is to make a substitution using Euler's identity: $e^{i\theta} = \cos(\theta) + i \sin(\theta)$. Using $\theta = \lambda x$, we change the following in the function:

$$\cos(\lambda x) = e^{i\lambda x} - i \sin(\lambda x)$$

Substituting this in the original function, we can split the function in equation (3.1.1) into a difference of terms:

$$e^{-x^2} (e^{i\lambda x} - i \sin(\lambda x)) = e^{-x^2+i\lambda x} - i e^{-x^2} \sin(\lambda x) \quad (3.1.2)$$

We would like the first term to be similar to the Gaussian exponential function, so we add and subtract a term b in the exponential and gather terms. For the next few lines we will only look at exponent for the first term in equation (3.1.2):

$$-x^2 + i\lambda x = -x^2 + i\lambda x + b - b \quad (3.1.3)$$

We would like to be able to write $-x^2 + i\lambda x + b$ as something of the form $x^2 + c^2 + 2xc = (x + c)^2$. So we factor out -1 from equation (3.1.3) on all terms except $-b$, which gives us $-(x^2 - i\lambda x - b) - b$. We then get $x^2 = x^2$, $c^2 = -b$, and $2xc = -i\lambda x$. We want to find a value for c :

$$\begin{aligned} c^2 = -b &\iff c = i\sqrt{b} \\ 2xc = -i\lambda x &\iff 2c = -i\lambda \iff c = -\frac{i\lambda}{2} \end{aligned}$$

Substituting $c = -\frac{i\lambda}{2}$ in $c = i\sqrt{b}$ yields for b :

$$\begin{aligned} i\sqrt{b} = c &= -\frac{i\lambda}{2} \\ \iff i\sqrt{b} &= -\frac{i\lambda}{2} \iff b = \frac{\lambda^2}{4} \end{aligned}$$

We can now substitute $b = \frac{\lambda^2}{4}$ in equation (3.1.3) and get $-(x + \frac{i\lambda}{2})^2$. Then we can write out the exponent on the first term in equation (3.1.2) as follows:

$$e^{-x^2 - \frac{\lambda^2}{4} - i\lambda x - \frac{\lambda^2}{4}} - i e^{-x^2} \sin(\lambda x) = e^{-(x + \frac{i\lambda}{2})^2} e^{-\frac{\lambda^2}{4}} - i e^{-x^2} \sin(\lambda x) \quad (3.1.4)$$

Substituting this into the function in equation (3.1.1), factoring out $e^{-\frac{\lambda^2}{4}}$ and using linearity of the integral gives us the following:

$$e^{-\frac{\lambda^2}{4}} \int_{-\infty}^{\infty} e^{-(x + \frac{i\lambda}{2})^2} dx - i \int_{-\infty}^{\infty} e^{-x^2} \sin(\lambda x) dx \quad (3.1.5)$$

For the first integral in equation (3.1.5) we use a substitution $u = x + \frac{i\lambda}{2}$ then $du = dx$. Then the first term becomes the Gaussian integral with respect to u which is equal to $\sqrt{\pi}$ when integrated on the real-line, [6]. The first term in equation (3.1.5) therefore becomes $\sqrt{\pi} e^{-\frac{\lambda^2}{4}}$.

The second term in equation (3.1.5) is the integral of the product of an even and odd function. The resulting function is therefore odd, [1, p. 308]. The integral over the real line therefore becomes 0, since it is a symmetric interval. The whole integral in equation (3.1.5) therefore becomes:

$$\sqrt{\pi} e^{-\frac{\lambda^2}{4}}$$

We now have a value for the integral of the function over the real line. We can also use the fact that the function is symmetric. This means that the integral from $-\infty$ to ∞ is equal to 2 times the integral from 0 to ∞ , [1, p. 308], that is:

$$\int_{-\infty}^{\infty} e^{-x^2} \cos(\lambda x) dx = 2 \int_0^{\infty} e^{-x^2} \cos(\lambda x) dx = \sqrt{\pi} e^{-\frac{\lambda^2}{4}} \quad (3.1.6)$$

We now have the integral of the one-dimensional function over the real line.

3.2 True solution of one-dimensional bounded integral

In order to numerically approximate the integral of the function we need to look at the integral on a interval $[-B, B]$, $B \in \mathbb{R}$. We again look at the integral of the function but now on the bounded interval:

$$\int_{-B}^B e^{-x^2} \cos(\lambda x) dx$$

Many of the steps are the same as the integral in section 3.1. The substitution using Euler's identity is the same as equation (3.1.2). We again add and subtract $b = \frac{\lambda^2}{4}$ in the exponent as in equation (3.1.4) and we again use linearity of the integral as in equation (3.1.5). The resulting integral becomes:

$$e^{-\frac{\lambda^2}{4}} \int_{-B}^B e^{-(x + \frac{i\lambda}{2})^2} dx - i \int_{-B}^B e^{-x^2} \sin(\lambda x) dx$$

As in equation (3.1.5), the integral $i \int_{-B}^B e^{-x^2} \sin(\lambda x) dx$ is the integral of an odd function on the symmetric interval $[-B, B]$, which is therefore 0. We are then left with the following integral:

$$e^{-\frac{\lambda^2}{4}} \int_{-B}^B e^{-(x + \frac{i\lambda}{2})^2} dx$$

First we separate the integral into a two integrals over the subintervals $[-B, 0]$ and $[0, B]$:

$$= e^{-\frac{\lambda^2}{4}} \left(\int_{-B}^0 e^{-(x+\frac{i\lambda}{2})^2} dx + \int_0^B e^{-(x+\frac{i\lambda}{2})^2} dx \right)$$

We then change the interval of integration from $[-B, 0]$ to $[0, -B]$ on the first integral and change the sign in front of it, [1, p. 308]:

$$= e^{-\frac{\lambda^2}{4}} \left(- \int_0^{-B} e^{-(x+\frac{i\lambda}{2})^2} dx + \int_0^B e^{-(x+\frac{i\lambda}{2})^2} dx \right)$$

We want to have terms that are similar to the error function $\text{erf}(z) = \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2} dt$. In order to do that we multiply and divide by $\frac{2}{\sqrt{\pi}}$ and substitute $x + \frac{i\lambda}{2} = u$. Doing this we also substitute the upper bound of the two integrals from $-B$ and B to $-B + \frac{i\lambda}{2}$ and $B + \frac{i\lambda}{2}$ respectively. That way $du = dx$ and we get the following:

$$= e^{-\frac{\lambda^2}{4}} \frac{\sqrt{\pi}}{2} \left(- \frac{2}{\sqrt{\pi}} \int_0^{-B+\frac{i\lambda}{2}} e^{-u^2} du + \frac{2}{\sqrt{\pi}} \int_0^{B+\frac{i\lambda}{2}} e^{-u^2} du \right)$$

We now rewrite the integrals into error functions with the upper bounds of the integrals as input:

$$= e^{-\frac{\lambda^2}{4}} \frac{\sqrt{\pi}}{2} \left(- \text{erf} \left(-B + \frac{i\lambda}{2} \right) + \text{erf} \left(B + \frac{i\lambda}{2} \right) \right)$$

In order to implement the integral solution of the function in MATLAB, we need to convert the error function into the imaginary error function, where the relation between the error function and imaginary error function is $\text{erfi}(z) = -i \text{erf}(iz)$, [3, p. 2]. To do this, we start by multiplying the integral by $1 = -i^2$ outside the parenthesis. We then pull out one i in each of the error function inputs. This can be done by changing $-B + \frac{i\lambda}{2} = i(iB + \frac{\lambda}{2})$ and $B + \frac{i\lambda}{2} = i(-iB + \frac{\lambda}{2})$. Finally, we pull one of the i 's into the parentheses and change the signs:

$$= e^{-\frac{\lambda^2}{4}} \frac{i\sqrt{\pi}}{2} \left(i \text{erf} \left(i \left(iB + \frac{\lambda}{2} \right) \right) - i \text{erf} \left(i \left(-iB + \frac{\lambda}{2} \right) \right) \right)$$

Now we use the identity for the imaginary error function:

$$= e^{-\frac{\lambda^2}{4}} \frac{i\sqrt{\pi}}{2} \left(- \text{erfi} \left(iB + \frac{\lambda}{2} \right) + \text{erfi} \left(-iB + \frac{\lambda}{2} \right) \right) \quad (3.2.1)$$

We now have a value for the integral of the function on the interval $[-B, B]$. As in section 3.1 the function is symmetric. Therefore, the integral from $-B$ to B is equal to 2 times the integral from 0 to B .

3.3 True solution of multi-dimensional integral

We would also like to find the integral of the multi-dimensional version of our function. Let $\boldsymbol{\lambda} = (\lambda_1, \lambda_2, \dots, \lambda_n)^T$, $\boldsymbol{x} = (x_1, x_2, \dots, x_n)^T$ where $\lambda_i, x_i \in \mathbb{R}$, $i = 1, \dots, n$. The multi-dimensional function is:

$$e^{-\sum_{i=1}^n x_i^2} \cos \left(\sum_{i=1}^n \lambda_i x_i \right)$$

We are interested in the integral over $\mathbb{R}^n$ of this function. We will find the integral for an arbitrary dimension n with a proof by induction over the dimension d . We will first show the base case for $d = 1$.

3.3.1 Base case: $d = 1$

We have already shown the base case back in section 3.1. That is:

$$\int_{-\infty}^{\infty} e^{-x^2} \cos(\lambda x) dx = \sqrt{\pi} e^{-\frac{\lambda^2}{4}} \quad (3.3.1)$$

This proves the base case for $d = 1$.

3.3.2 Inductive hypothesis

It might be difficult to discern a particular pattern for the integral for a arbitrary dimension just based on the one-dimensional case. So we will first look at the case when $d = 2$. We will use the base case for $d = 1$ to get the integral for the two-dimensional case:

$$\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} e^{-(x_1^2+x_2^2)} \cos(\lambda_1 x_1 + \lambda_2 x_2) dx_1 dx_2 \quad (3.3.2)$$

The exponential function can be separated into two terms, where one term is independent of the current variable of integration.

$$= \int_{-\infty}^{\infty} e^{-x_2^2} \int_{-\infty}^{\infty} e^{-x_1^2} \cos(\lambda_1 x_1 + \lambda_2 x_2) dx_1 dx_2 \quad (3.3.3)$$

The cosine function cannot be separated the same way. We can however use a trigonometric identity to split the function. If we have two values $\alpha, \beta \in \mathbb{R}$ it holds that $\cos(\alpha + \beta) = \cos(\alpha)\cos(\beta) - \sin(\alpha)\sin(\beta)$ [1, p. 51]. Setting $\alpha = \lambda_1 x_1$ and $\beta = \lambda_2 x_2$ gives us:

$$= \int_{-\infty}^{\infty} e^{-x_2^2} \int_{-\infty}^{\infty} e^{-x_1^2} \left(\cos(\lambda_1 x_1) \cos(\lambda_2 x_2) - \sin(\lambda_1 x_1) \sin(\lambda_2 x_2) \right) dx_1 dx_2 \quad (3.3.4)$$

Multiplying each term in the parentheses by $e^{-x_1^2}$ and by linearity of the integral, we can split the integral into two integrals. Furthermore, $\sin(\lambda_2 x_2)$ and $\cos(\lambda_2 x_2)$ are constant w.r.t. to x_1 , so we can pull them out of the integrals. Equation (3.3.4) can then be rewritten like this:

$$= \int_{-\infty}^{\infty} e^{-x_2^2} \left(\cos(\lambda_2 x_2) \int_{-\infty}^{\infty} e^{-x_1^2} \cos(\lambda_1 x_1) dx_1 - \sin(\lambda_2 x_2) \int_{-\infty}^{\infty} e^{-x_1^2} \sin(\lambda_1 x_1) dx_1 \right) dx_2 \quad (3.3.5)$$

The function $e^{-x_1^2} \sin(\lambda_1 x_1)$ is odd, so the integral of it becomes zero. We can then use what we derived in section 3.1:

$$= \int_{-\infty}^{\infty} e^{-x_2^2} \cos(\lambda_2 x_2) \left(\int_{-\infty}^{\infty} e^{-x_1^2} \cos(\lambda_1 x_1) dx_1 \right) dx_2 \quad (3.3.6)$$

$$= \sqrt{\pi} e^{-\frac{\lambda_1^2}{4}} \int_{-\infty}^{\infty} e^{-x_2^2} \cos(\lambda_2 x_2) dx_2 \quad (3.3.7)$$

Using what we know from the one-dimensional case again:

$$= \sqrt{\pi} e^{-\frac{\lambda_1^2}{4}} \sqrt{\pi} e^{-\frac{\lambda_2^2}{4}} = \pi e^{-\frac{\lambda_1^2 + \lambda_2^2}{4}}$$

Now we have the solution to the two-dimensional integral. From this result, it seems like the pattern for the d -dimensional integral is that it is equal to $(\sqrt{\pi})^d e^{-\frac{1}{4} \sum_{i=1}^d \lambda_i^2}$. This will be our inductive hypothesis.

3.3.3 Induction step

We have our inductive hypothesis for the d -dimensional integral:

$$\int_{\mathbb{R}^d} e^{-\sum_{i=1}^d x_i^2} \cos\left(\sum_{i=1}^d \lambda_i x_i\right) dx_1 \dots dx_d = (\sqrt{\pi})^d e^{-\frac{1}{4} \sum_{i=1}^d \lambda_i^2} \quad (3.3.8)$$

We want to use this to show that:

$$\int_{\mathbb{R}^{d+1}} e^{-\sum_{i=1}^{d+1} x_i^2} \cos\left(\sum_{i=1}^{d+1} \lambda_i x_i\right) dx_1 \dots dx_{d+1} = (\sqrt{\pi})^{d+1} e^{-\frac{1}{4} \sum_{i=1}^{d+1} \lambda_i^2} \quad (3.3.9)$$

Looking at equation (3.3.9), we want to isolate the terms involving x_{d+1} . First, we can pull $e^{-x_{d+1}^2}$ out of the exponential function. This term is constant w.r.t. to $x_1, x_2, \dots, x_d$. We can then use the same trigonometric identity as in equation (3.3.4) with $\alpha = \lambda_{d+1}x_{d+1}$ and $\beta = \sum_{i=1}^d \lambda_i x_i$. We then get the following:

$$\begin{aligned} &= \int_{-\infty}^{\infty} e^{-x_{d+1}^2} \int_{-\infty}^{\infty} \dots \int_{-\infty}^{\infty} e^{-\sum_{i=1}^d x_i^2} \left(\cos(\lambda_{d+1}x_{d+1}) \cos\left(\sum_{i=1}^d \lambda_i x_i\right) \right. \\ &\quad \left. - \sin(\lambda_{d+1}x_{d+1}) \sin\left(\sum_{i=1}^d \lambda_i x_i\right) \right) dx_1 \dots dx_d \end{aligned} \quad (3.3.10)$$

As in the two-dimensional case, we can split the integral via linearity. As in equation (3.3.5), the integral of $e^{-\sum_{i=1}^d x_i^2} \sin\left(\sum_{i=1}^d \lambda_i x_i\right)$ is 0 since it is an odd function. Using the fact that $\cos(\lambda_{d+1}x_{d+1})$ is constant w.r.t. to $x_1, x_2, \dots, x_d$, we can factor the term out of the integrals. We then get the following:

$$= \int_{-\infty}^{\infty} e^{-x_{d+1}^2} \cos(\lambda_{d+1}x_{d+1}) dx_{d+1} \int_{-\infty}^{\infty} \dots \int_{-\infty}^{\infty} e^{-\sum_{i=1}^d x_i^2} \cos\left(\sum_{i=1}^d \lambda_i x_i\right) dx_1 \dots dx_d \quad (3.3.11)$$

Now we use the inductive hypothesis from equation (3.3.8) to evaluate the integral in equation (3.3.11) and use the base case from equation (3.3.1) to evaluate the remaining term:

$$\begin{aligned} &= \int_{-\infty}^{\infty} e^{-x_{d+1}^2} \cos(\lambda_{d+1}x_{d+1}) dx_{d+1} \left((\sqrt{\pi})^d e^{-\frac{1}{4} \sum_{i=1}^d \lambda_i^2} \right) \\ &= \left((\sqrt{\pi})^d e^{-\frac{1}{4} \sum_{i=1}^d \lambda_i^2} \right) \int_{-\infty}^{\infty} e^{-x_{d+1}^2} \cos(\lambda_{d+1}x_{d+1}) dx_{d+1} \\ &= (\sqrt{\pi})^{d+1} e^{-\frac{1}{4} \sum_{i=1}^{d+1} \lambda_i^2} \end{aligned}$$

We have thus shown that:

$$\int_{\mathbb{R}^{d+1}} e^{-\sum_{i=1}^{d+1} x_i^2} \cos\left(\sum_{i=1}^{d+1} \lambda_i x_i\right) dx_1 \dots dx_{d+1} = (\sqrt{\pi})^{d+1} e^{-\frac{1}{4} \sum_{i=1}^{d+1} \lambda_i^2}$$

This gives us the formula for the integral of the multi-dimensional function.

3.4 True solution of multi-dimensional bounded integral

As in section 3.2, we also want the integral when the function is restricted to a subset of $\mathbb{R}^d$. For the multi-dimensional case, we consider vectors of bound values in each dimension $\mathbf{B} = (B_1, B_2, \dots, B_d)^T$ and a vector of λ values $\boldsymbol{\lambda} = (\lambda_1, \lambda_2, \dots, \lambda_d)^T$. The domain of integration is $\Omega = [-B_1, B_1] \times [-B_2, B_2] \times \dots \times [-B_d, B_d]$. The integral of the function over Ω can be written as:

$$\int_{\Omega} e^{-\sum_{i=1}^d x_i^2} \cos\left(\sum_{i=1}^d \lambda_i x_i\right) dx_1 \dots dx_d = \int_{-B_1}^{B_1} \int_{-B_2}^{B_2} \dots \int_{-B_d}^{B_d} e^{-\sum_{i=1}^d x_i^2} \cos\left(\sum_{i=1}^d \lambda_i x_i\right) dx_1 \dots dx_d \quad (3.4.1)$$

The method of integration is similar to the integral of the function over $\mathbb{R}^d$. We can use the results from equations (3.3.10) and (3.3.11). The trigonometric identity is used the same way as in equation (3.3.10) and since we integrate over symmetric intervals in each dimension, the integral of $e^{-x_i^2} \sin(\lambda_i x_i)$ is still 0 for all $i = 1, \dots, d$. We can therefore separate the integral like this:

$$\int_{-B_1}^{B_1} e^{-x_1^2} \cos(\lambda_1 x_1) dx_1 \int_{-B_2}^{B_2} e^{-x_2^2} \cos(\lambda_2 x_2) dx_2 \dots \int_{-B_d}^{B_d} e^{-x_d^2} \cos(\lambda_d x_d) dx_d \quad (3.4.2)$$

The integral has become the product of integrals of the one-dimensional function in each dimension. We can therefore use the earlier result from equation (3.2.1) to get the solution of the integral, since equation (3.4.2) is as product of one-dimensional integrals. Equation (3.4.2) then becomes:

$$= e^{-\frac{1}{4} \sum_{k=1}^d \lambda_k^2} \frac{i\sqrt{\pi}}{2} \prod_{k=1}^d \left(-\operatorname{erfi}\left(iB_k + \frac{\lambda_k}{2}\right) + \operatorname{erfi}\left(-iB_k + \frac{\lambda_k}{2}\right) \right) \quad (3.4.3)$$

Since the expression involves the imaginary unit i , we will index the values of λ and B over k to avoid confusion. We now have a formula for the true integral of the multi-dimensional function on a bounded d -dimensional domain.

4 Extending Newton-Cotes methods to two dimensions

In order to numerically approximate the integral of a higher dimensional function, we need to extend the Newton-Cotes methods to multiple dimensions. We will be extending the already discussed Newton-Cotes methods to two dimensions, starting with the midpoint rule. The following derivations are based on [2].

4.1 Composite midpoint rule

As discussed in section 2.3.1, the formula for the composite midpoint rule in one dimension is:

$$\int_a^b f(x) dx \approx h \sum_{i=1}^n f(w_i) \quad (4.1.1)$$

In order to integrate our function in two dimensions, we now wish to generalise the midpoint rule to two dimensions. The following equations will use $x_{i-1} + \frac{h}{2}$ instead of w_i . This will be the case for both the x and y values and the step sizes h_x and h_y . We consider the integral of a two-dimensional function over the intervals $[x_0, x_n]$ and $[y_0, y_m]$, $n, m \in \mathbb{N}$ with step sizes $h_x = \frac{(x_n - x_0)}{n}$ and $h_y = \frac{(y_m - y_0)}{m}$:

$$I_{2D} = \int_{x_0}^{x_n} \int_{y_0}^{y_m} f(x, y) dy dx = \int_{x_0}^{x_n} g(x) dx \quad (4.1.2)$$

We consider the inner term $g(x)$ which can be approximated with the midpoint rule:

$$g(x) = \int_{y_0}^{y_m} f(x, y) dy \approx h_y \sum_{j=1}^m f\left(x, y_{j-1} + \frac{h_y}{2}\right)$$

Putting the approximation for $g(x)$ into equation (4.1.2) gives the following:

$$I_{2D} \approx \int_{x_0}^{x_n} h_y \sum_{j=1}^m f\left(x, y_{j-1} + \frac{h_y}{2}\right) dx \quad (4.1.3)$$

We can now factor out h_y and use linearity of integrals to rewrite equation (4.1.3) as a sum of integrals:

$$= h_y \sum_{j=1}^m \int_{x_0}^{x_n} f\left(x, y_{j-1} + \frac{h_y}{2}\right) dx \quad (4.1.4)$$

Now we apply the composite midpoint rule to each integral in equation (4.1.4). This gives us the following:

$$\approx h_y h_x \sum_{j=1}^m \sum_{i=1}^n f\left(x_{i-1} + \frac{h_x}{2}, y_{j-1} + \frac{h_y}{2}\right) \quad (4.1.5)$$

We now have the formula for the 2D composite midpoint rule.

As for the computational complexity of the method, we count the number of times the function needs to be evaluated. Because it uses a double sum, it needs to evaluate $n \cdot m$ different terms. It therefore has a running time of $\mathcal{O}(nm)$. If the number of steps is equal in each dimension such that $n = m = N$, the computational complexity of the method is $\mathcal{O}(N^2)$.

4.1.1 Error of 2D composite midpoint rule

As mentioned in section 2.3.1, the error term for the midpoint rule in one dimension is $\frac{(b-a)h^2}{24} f''(c)$ for some $c \in [a, b]$. Therefore, if we want equality between the midpoint rule and the true solution of the integral of a function $f(x)$, we write the following:

$$\int_a^b f(x) dx = h \sum_{i=1}^n f\left(x_{i-1} + \frac{h}{2}\right) + \frac{(b-a)h^2}{24} f''(c) \quad (4.1.6)$$

In order to assess the numerical accuracy of the 2D composite midpoint rule, we want to derive a similar error term. First, we again consider the two-dimensional integral of $f(x, y)$, where $x_0 < x_1 < \dots < x_n$ and $y_0 < y_1 < \dots < y_m$:

$$\int_{x_0}^{x_n} \int_{y_0}^{y_m} f(x, y) dy dx = \int_{x_0}^{x_n} g(x) dx \quad (4.1.7)$$

Where $g(x) = \int_{y_0}^{y_m} f(x, y) dy$. We will now write out $g(x)$ using equation (4.1.6). Since $f(x, y)$ is a multi-variate function, we cannot simply take the second order total derivative. Since we want to use the midpoint rule to integrate the function w.r.t. y first, we will instead take the second order partial derivative w.r.t. to y :

$$g(x) = \int_{y_0}^{y_m} f(x, y) dy = h_y \sum_{j=1}^m f\left(x, y_{j-1} + \frac{h_y}{2}\right) + \frac{(y_m - y_0)h_y^2}{24} \cdot \frac{\partial^2 f(x, c_{y_1})}{\partial y^2} \quad (4.1.8)$$

With $c_{y_1} \in [y_0, y_m]$. Every time we use constants c_{x_i} or c_{y_j} in the error terms in the following sections, it should be noted that they refer to values in the intervals of values of x or y . They all come from the error terms from [5]. Setting $g(x)$ to (4.1.8) in equation (4.1.7) and applying the midpoint rule to both the integral of the function and the error term gives us the following:

$$\int_{x_0}^{x_n} h_y \sum_{j=1}^m f\left(x, y_{j-1} + \frac{h_y}{2}\right) + \frac{(y_m - y_0)h_y^2}{24} \cdot \frac{\partial^2 f(x, c_{y_1})}{\partial y^2} dx$$

Using the linearity of the integral on the sum and the error term in the parenthesis and factoring out their respective constants h_y and $\frac{(y_m - y_0)h_y^2}{24}$ gives the following:

$$= h_y \sum_{j=1}^m \int_{x_0}^{x_n} f\left(x, y_{j-1} + \frac{h_y}{2}\right) dx + \frac{(y_m - y_0)h_y^2}{24} \int_{x_0}^{x_n} \frac{\partial^2 f(x, c_{y_1})}{\partial y^2} dx \quad (4.1.9)$$

Now we apply the midpoint rule to both terms. The error will have a partial derivative w.r.t. to x now, since that is the variable of integration. We also get a fourth order partial derivative that will be differentiated twice w.r.t y and twice w.r.t x when we apply the midpoint rule to the integral in the error term. We then get the following:

$$\begin{aligned} &= h_y \sum_{j=1}^m \left(h_x \sum_{i=1}^n f\left(x_{i-1} + \frac{h_x}{2}, y_{j-1} + \frac{h_y}{2}\right) + \frac{(x_n - x_0)h_x^2}{24} \cdot \frac{\partial^2 f(c_{x_1}, y_{j-1} + \frac{h_y}{2})}{\partial x^2} \right) \\ &+ \frac{(y_m - y_0)h_y^2}{24} \left(h_x \sum_{i=1}^n \frac{\partial^2 f(x_{i-1} + \frac{h_x}{2}, c_{y_1})}{\partial y^2} + \frac{(x_n - x_0)h_x^2}{24} \cdot \frac{\partial^4 f(c_{x_2}, c_{y_1})}{\partial y^2 \partial x^2} \right) \end{aligned} \quad (4.1.10)$$

With some $c_{x_1}, c_{x_2} \in [x_0, x_n]$. We can separate the first term in equation (4.1.10) into two sums, the first of which is just the composite midpoint rule as in equation (4.1.5). Since it does not change the remaining error terms, we will omit it from the following equations and only focus on the error terms. Removing the 2D composite midpoint rule and writing out the rest of equation (4.1.10) gives us the following sum of error terms for the 2D composite midpoint rule:

$$\begin{aligned} &\frac{(x_n - x_0)h_x^2 h_y}{24} \sum_{j=1}^m \frac{\partial^2 f(c_{x_1}, y_{j-1} + \frac{h_y}{2})}{\partial x^2} + \frac{(y_m - y_0)h_y^2 h_x}{24} \sum_{i=1}^n \frac{\partial^2 f(x_{i-1} + \frac{h_x}{2}, c_{y_1})}{\partial y^2} \\ &+ \frac{(y_m - y_0)(x_n - x_0)h_y^2 h_x^2}{576} \cdot \frac{\partial^4 f(c_{x_2}, c_{y_1})}{\partial y^2 \partial x^2} \end{aligned} \quad (4.1.11)$$

We will now use the Generalized Intermediate Value Theorem (2.2.1) in order to remove the sums on the first two error terms. For the sums of second order partial derivatives w.r.t to first x and then y , we let each a_j and a_i , $j = 1, \dots, m, i = 1, \dots, n$ be equal to 1 for the setting of theorem (2.2.1). Since each of these coefficients is equal to 1, the sum of terms will be equal to m and n for the two sums respectively. And the functions will be evaluated at new values in between the intervals. From there, we will use the fact that $m = \frac{(y_m - y_0)}{h_y}$ and $n = \frac{(x_n - x_0)}{h_x}$ and cancel one term for h_y and h_x in the two terms respectively. Doing so gives us the following error terms:

$$\begin{aligned} &\frac{(x_n - x_0)(y_m - y_0)h_x^2}{24} \cdot \frac{\partial^2 f(c_{x_1}, c_{y_2})}{\partial x^2} + \frac{(x_n - x_0)(y_m - y_0)h_y^2}{24} \cdot \frac{\partial^2 f(c_{x_3}, c_{y_1})}{\partial y^2} \\ &+ \frac{(y_m - y_0)(x_n - x_0)h_y^2 h_x^2}{576} \cdot \frac{\partial^4 f(c_{x_2}, c_{y_1})}{\partial y^2 \partial x^2} \end{aligned} \quad (4.1.12)$$

With $c_{x_3} \in [x_0, x_n]$ and $c_{y_2} \in [y_0, y_m]$. The first two terms in the sum feature h_x^2 and h_y^2 respectively, while the final term features the product $h_x^2 h_y^2$. Since h_x and h_y are values that will go to 0, the terms with h_x^2 and h_y^2 are going to decrease in value at a slower rate than the term with $h_x^2 h_y^2$. The error terms involving just h_x^2 and h_y^2 will therefore go to 0 at a slower rate than the error term with $h_x^2 h_y^2$. The order of the method is therefore determined by those larger error terms. The order of the method is therefore $\mathcal{O}(h_x^2 + h_y^2)$ since the error decreases at a rate of 2 in both dimensions. As discussed in section 2.3.1 the order of the one-dimensional composite midpoint rule is 2, meaning that the order of the error does not change when the method is extended to two dimensions.

We want to compare the numerical accuracy of the 2D composite midpoint rule with other methods of numerical integration. We therefore need the order of the error to be based on the number of steps instead of the step size. Using the fact that the step size and number of steps are inversely proportional, $h_x^2 = \frac{(x_n - x_0)^2}{n^2}$ and $h_y^2 = \frac{(y_m - y_0)^2}{m^2}$. We can substitute these into equation (4.1.12) and get:

$$\begin{aligned} & \frac{(x_n - x_0)^3 (y_m - y_0)}{24n^2} \cdot \frac{\partial^2 f(c_{x_1}, c_{y_2})}{\partial x^2} + \frac{(x_n - x_0)(y_m - y_0)^3}{24m^2} \cdot \frac{\partial^2 f(c_{x_3}, c_{y_1})}{\partial y^2} \\ & + \frac{(y_m - y_0)^3 (x_n - x_0)^3}{576n^2 m^2} \cdot \frac{\partial^4 f(c_{x_2}, c_{y_1})}{\partial y^2 \partial x^2} \end{aligned} \quad (4.1.13)$$

With this we can see that the error of the 2D composite midpoint rule has an error of order $\mathcal{O}(n^{-2} + m^{-2})$. With an equal number of steps in each dimension, $n = m = N$, it equates to $\mathcal{O}(N^{-2})$.

4.2 Composite trapezoid rule

As with the midpoint rule, we also want to derive the composite trapezoid rule for a two-dimensional integral. As discussed in section 2.3.2, the formula for the composite trapezoid rule in one dimension is:

$$\int_a^b f(x) dx \approx \frac{h}{2} \left(f(x_0) + f(x_n) + 2 \sum_{i=1}^{n-1} f(x_i) \right) \quad (4.2.1)$$

The intervals and step sizes are the same as in section 4.1. We then consider the integral of a two-dimensional function:

$$I_{2D} = \int_{x_0}^{x_n} \int_{y_0}^{y_m} f(x, y) dy dx = \int_{x_0}^{x_n} g(x) dx \quad (4.2.2)$$

Writing out $g(x)$ using the trapezoid rule, (4.2.1), gives the following:

$$g(x) = \int_{y_0}^{y_m} f(x, y) dy \approx \frac{h_y}{2} \left(f(x, y_0) + f(x, y_m) + 2 \sum_{j=1}^{m-1} f(x, y_j) \right)$$

Putting the approximation for $g(x)$ into equation (4.2.2) gives us:

$$I_{2D} \approx \int_{x_0}^{x_n} \frac{h_y}{2} \left(f(x, y_0) + f(x, y_m) + 2 \sum_{j=1}^{m-1} f(x, y_j) \right) dx \quad (4.2.3)$$

By linearity of integral, this can be rewritten as three separate integrals:

$$= \frac{h_y}{2} \left(\int_{x_0}^{x_n} f(x, y_0) dx + \int_{x_0}^{x_n} f(x, y_m) dx + 2 \sum_{j=1}^{m-1} \int_{x_0}^{x_n} f(x, y_j) dx \right) \quad (4.2.4)$$

Using the composite trapezoid rule (4.2.1) on each of the three integrals in equation (4.2.4), we get the following:

$$\approx \frac{h_y}{2} \left(\begin{aligned} & \frac{h_x}{2} \left(f(x_0, y_0) + f(x_n, y_0) + 2 \sum_{i=1}^{n-1} f(x_i, y_0) \right) \\ & + \frac{h_x}{2} \left(f(x_0, y_m) + f(x_n, y_m) + 2 \sum_{i=1}^{n-1} f(x_i, y_m) \right) \\ & + 2 \sum_{j=1}^{m-1} \frac{h_x}{2} \left(f(x_0, y_j) + f(x_n, y_j) + 2 \sum_{i=1}^{n-1} f(x_i, y_j) \right) \end{aligned} \right) \quad (4.2.5)$$

We can simplify equation (4.2.5). First, we can pull $\frac{h_x}{2}$ out of the parentheses. We can then gather all terms that are not in a sum, that is to say $f(x_0, y_0)$, $f(x_n, y_0)$, $f(x_0, y_m)$, and $f(x_n, y_m)$. Furthermore, we can gather the single sums into two sums over indices i and j respectively. Finally, we have a double sum over both i and j that we can gather separately from the other terms. Doing this gives us:

$$= \frac{h_y h_x}{4} \left(\begin{aligned} & f(x_0, y_0) + f(x_n, y_0) + f(x_0, y_m) + f(x_n, y_m) \\ & + 2 \sum_{i=1}^{n-1} [f(x_i, y_0) + f(x_i, y_m)] \\ & + 2 \sum_{j=1}^{m-1} [f(x_0, y_j) + f(x_n, y_j)] \\ & + 4 \sum_{j=1}^{m-1} \sum_{i=1}^{n-1} f(x_i, y_j) \end{aligned} \right) \quad (4.2.6)$$

We now have a formula for the 2D composite trapezoid rule. Next we would like to derive the computational complexity of the method. There are four terms that are not part of a sum, so they contribute constant work regardless of n and m . There are two terms that sum $2(n-1)$ and $2(m-1)$ different terms respectively as well as a double sum that has $(m-1)(n-1)$ different terms. This means that there is a total of $4 + 2(n-1) + 2(m-1) + (n-1)(m-1) = nm + n + m + 1$ separate terms to be evaluated. This means the computational complexity is $\mathcal{O}(nm)$, and if we use the same number of steps in each dimension such that $n = m = N$, the computational complexity is $\mathcal{O}(N^2)$.

4.2.1 Error of 2D composite trapezoid rule

Similar to the midpoint rule, we would also like to derive the order of accuracy of the 2D composite trapezoid rule. The method for deriving the error terms is similar to the 2D midpoint rule, while the general error term for the one-dimensional trapezoid rule is $-\frac{(b-a)h^2}{12}f''(c)$, $c \in [a, b]$, as mentioned in section 2.3.2. We again write out the 2D integral and apply the composite trapezoid rule first on the integral w.r.t. to y and then again on the resulting expression.

$$I_{2D} = \int_{x_0}^{x_n} \int_{y_0}^{y_m} f(x, y) dy dx = \int_{x_0}^{x_n} g(x) dx \quad (4.2.7)$$

Where $g(x)$ is defined as follows:

$$g(x) = \int_{y_0}^{y_m} f(x, y) dy$$

Applying the composite trapezoid rule to $g(x)$ in equation (4.2.7) and including the error term gives us the following:

$$\int_{x_0}^{x_n} \frac{h_y}{2} \left(f(x, y_0) + f(x, y_m) + 2 \sum_{j=1}^{m-1} f(x, y_j) \right) - \frac{(y_m - y_0)h_y^2}{12} \cdot \frac{\partial^2 f(x, c_{y_1})}{\partial y^2} dx \quad (4.2.8)$$

Where $c_{y_1} \in [y_0, y_m]$. Similar to what we did for the midpoint rule, we take the second order partial derivative w.r.t. y , since $f(x, y)$ is a multivariate function. Factoring out $\frac{h_y}{2}$, using the linearity of integrals, and factoring out $-\frac{(y_m - y_0)h_y^2}{12}$ gives the following.

$$\begin{aligned} &= \frac{h_y}{2} \left(\int_{x_0}^{x_n} f(x, y_0) dx + \int_{x_0}^{x_n} f(x, y_m) dx + 2 \sum_{j=1}^{m-1} \int_{x_0}^{x_n} f(x, y_j) dx \right) \\ &\quad - \frac{(y_m - y_0)h_y^2}{12} \int_{x_0}^{x_n} \frac{\partial^2 f(x, c_{y_1})}{\partial y^2} dx \end{aligned} \quad (4.2.9)$$

We now apply the composite trapezoid rule on all integrals in equation (4.2.9):

$$\begin{aligned} &= \frac{h_y}{2} \left(\frac{h_x}{2} \left(f(x_0, y_0) + f(x_n, y_0) + 2 \sum_{i=1}^{n-1} f(x_i, y_0) \right) - \frac{(x_n - x_0)h_x^2}{12} \cdot \frac{\partial^2 f(c_{x_1}, y_0)}{\partial x^2} \right. \\ &\quad + \frac{h_x}{2} \left(f(x_0, y_m) + f(x_n, y_m) + 2 \sum_{i=1}^{n-1} f(x_i, y_m) \right) - \frac{(x_n - x_0)h_x^2}{12} \cdot \frac{\partial^2 f(c_{x_2}, y_m)}{\partial x^2} \\ &\quad \left. + 2 \sum_{j=1}^{m-1} \left(\frac{h_x}{2} \left(f(x_0, y_j) + f(x_n, y_j) + 2 \sum_{i=1}^{n-1} f(x_i, y_j) \right) - \frac{(x_n - x_0)h_x^2}{12} \cdot \frac{\partial^2 f(c_{x_3}, y_j)}{\partial x^2} \right) \right) \\ &\quad - \frac{(y_m - y_0)h_y^2}{12} \left(\frac{h_x}{2} \left(\frac{\partial^2 f(x_0, c_{y_1})}{\partial y^2} + \frac{\partial^2 f(x_n, c_{y_1})}{\partial y^2} + 2 \sum_{i=1}^{n-1} \frac{\partial^2 f(x_i, c_{y_1})}{\partial y^2} \right) - \frac{(x_n - x_0)h_x^2}{12} \cdot \frac{\partial^4 f(c_{x_4}, c_{y_1})}{\partial y^2 \partial x^2} \right) \end{aligned} \quad (4.2.10)$$

With $c_{x_1}, c_{x_2}, c_{x_3}, c_{x_4} \in [x_0, x_n]$. The fourth order partial derivative is from the error when applying the composite trapezoid rule on $\frac{\partial^2 f(x, c_{y_1})}{\partial y^2}$ w.r.t. to x . If we multiply each term in the first parenthesis by $\frac{h_y}{2}$ we can gather all the error terms and all the terms that constitute the 2D composite trapezoid rule (4.2.6) we derived earlier separately. We will therefore only be looking at the error terms since the terms in the 2D composite trapezoid rule will not affect the error. On the error term outside the parenthesis in equation (4.2.10) we multiply $-\frac{(y_m - y_0)h_y^2}{12}$ on the terms in the parenthesis. This gives us the following:

$$\begin{aligned}
 & - \frac{(x_n - x_0)h_x^2 h_y}{24} \cdot \frac{\partial^2 f(c_{x_1}, y_0)}{\partial x^2} - \frac{(x_n - x_0)h_x^2 h_y}{24} \cdot \frac{\partial^2 f(c_{x_2}, y_m)}{\partial x^2} - \frac{(x_n - x_0)h_x^2 h_y}{24} 2 \sum_{j=1}^{m-1} \frac{\partial^2 f(c_{x_3}, y_j)}{\partial x^2} \\
 & - \frac{(y_m - y_0)h_y^2 h_x}{24} \cdot \frac{\partial^2 f(x_0, c_{y_1})}{\partial y^2} - \frac{(y_m - y_0)h_y^2 h_x}{24} \cdot \frac{\partial^2 f(x_n, c_{y_1})}{\partial y^2} - \frac{(y_m - y_0)h_y^2 h_x}{24} 2 \sum_{i=1}^{n-1} \frac{\partial^2 f(x_i, c_{y_1})}{\partial y^2} \\
 & + \frac{(y_m - y_0)(x_n - x_0)h_y^2 h_x^2}{144} \cdot \frac{\partial^4 f(c_{x_4}, c_{y_1})}{\partial y^2 \partial x^2}
 \end{aligned} \tag{4.2.11}$$

We will now pull the first three terms involving $-\frac{(x_n - x_0)h_x^2 h_y}{24}$ into a parentheses and do the same for the first three terms involving $-\frac{(y_m - y_0)h_y^2 h_x}{24}$. This gives us the following:

$$\begin{aligned}
 & = - \frac{(x_n - x_0)h_x^2 h_y}{24} \left(\frac{\partial^2 f(c_{x_1}, y_0)}{\partial x^2} + \frac{\partial^2 f(c_{x_2}, y_m)}{\partial x^2} + 2 \sum_{j=1}^{m-1} \frac{\partial^2 f(c_{x_3}, y_j)}{\partial x^2} \right) \\
 & - \frac{(y_m - y_0)h_y^2 h_x}{24} \left(\frac{\partial^2 f(x_0, c_{y_1})}{\partial y^2} + \frac{\partial^2 f(x_n, c_{y_1})}{\partial y^2} + 2 \sum_{i=1}^{n-1} \frac{\partial^2 f(x_i, c_{y_1})}{\partial y^2} \right) \\
 & + \frac{(y_m - y_0)(x_n - x_0)h_y^2 h_x^2}{144} \cdot \frac{\partial^4 f(c_{x_4}, c_{y_1})}{\partial y^2 \partial x^2}
 \end{aligned} \tag{4.2.12}$$

We will now apply theorem (2.2.1) on the terms in the parentheses. For the terms with x or y either indexed 0, n or m , the a_i or a_j coefficients will be 1, while for the terms in the sums, that is to say $i = 1, \dots, n-1$ or $j = 1, \dots, m-1$ the coefficients will be equal to 2. Then theorem (2.2.1) says that there exists values $c_{x_5}, c_{x_6} \in [x_0, x_n]$ and $c_{y_2} \in [y_0, y_m]$ such that the following holds:

$$\begin{aligned}
 & - \frac{(x_n - x_0)h_x^2 h_y}{24} \left((1 + 1 + 2(m-1)) \frac{\partial^2 f(c_{x_5}, c_{y_2})}{\partial x^2} \right) - \frac{(y_m - y_0)h_y^2 h_x}{24} \left((1 + 1 + 2(n-1)) \frac{\partial^2 f(c_{x_6}, c_{y_1})}{\partial y^2} \right) \\
 & + \frac{(y_m - y_0)(x_n - x_0)h_y^2 h_x^2}{144} \cdot \frac{\partial^4 f(c_{x_4}, c_{y_1})}{\partial x^2 \partial y^2}
 \end{aligned} \tag{4.2.13}$$

$1 + 1 + 2(m-1) = 2m$ and $1 + 1 + 2(n-1) = 2n$. As we did with the midpoint rule, we now use the fact that $m = \frac{(y_m - y_0)}{h_y}$ and $n = \frac{(x_n - x_0)}{h_x}$ to cancel one of the step sizes in each the first two terms in equation (4.2.13):

$$\begin{aligned}
 & = - \frac{(x_n - x_0)(y_m - y_0)h_x^2}{12} \cdot \frac{\partial^2 f(c_{x_5}, c_{y_2})}{\partial x^2} - \frac{(x_n - x_0)(y_m - y_0)h_y^2}{12} \cdot \frac{\partial^2 f(c_{x_6}, c_{y_1})}{\partial y^2} \\
 & + \frac{(y_m - y_0)(x_n - x_0)h_y^2 h_x^2}{144} \cdot \frac{\partial^4 f(c_{x_4}, c_{y_1})}{\partial y^2 \partial x^2}
 \end{aligned} \tag{4.2.14}$$

The first two error terms are of order $\mathcal{O}(h_x^2)$ and $\mathcal{O}(h_y^2)$ while the third term is of order $\mathcal{O}(h_x^2 h_y^2)$. As with the midpoint rule, this means that the first two error terms go to 0 at a slower rate than the last error term. It is therefore these error terms that determine the order of accuracy of the method. The 2D composite trapezoid rule is therefore accurate of order 2.

Then as we did for the midpoint rule, we need the order of the method to be based on the number of steps. Substituting $h_x^2 = \frac{(x_n - x_0)^2}{n^2}$ and $h_y^2 = \frac{(y_m - y_0)^2}{m^2}$ in equation (4.2.14) we get:

$$\begin{aligned} & - \frac{(x_n - x_0)^3(y_m - y_0)}{12n^2} \cdot \frac{\partial^2 f(c_{x_5}, c_{y_2})}{\partial x^2} - \frac{(x_n - x_0)(y_m - y_0)^3}{12m^2} \cdot \frac{\partial^2 f(c_{x_6}, c_{y_1})}{\partial y^2} \\ & + \frac{(y_m - y_0)^3(x_n - x_0)^3}{144n^2m^2} \cdot \frac{\partial^4 f(c_{x_4}, c_{y_1})}{\partial y^2 \partial x^2} \end{aligned}$$

As argued for the order of the error based on step size, the first two terms will be the larger w.r.t. n and m than the term with both n and m . This means that the order of the error based on the number of steps will be $\mathcal{O}(n^{-2} + m^{-2})$. And if the same number of steps is used in both dimensions such that $n = m = N$, the order of accuracy can be rewritten as $\mathcal{O}(N^{-2})$.

4.3 Composite Simpson's rule

We also want to generalize the composite Simpson's rule to two dimensions, especially since it has a higher order of accuracy than the composite midpoint and the composite trapezoid rules in one dimension. As discussed in section 2.3.3, the formula for the composite Simpson's rule in one dimension is:

$$\int_a^b f(x) dx \approx \frac{h}{3} \left(f(x_0) + f(x_{2n}) + 4 \sum_{i=1}^n f(x_{2i-1}) + 2 \sum_{i=1}^{n-1} f(x_{2i}) \right) \quad (4.3.1)$$

We again consider the two-dimensional integral as in the cases with the 2D composite midpoint and composite trapezoid rules. However, the bounds of integration are now $[x_0, x_{2n}]$ and $[y_0, y_{2m}]$. This also means that $h_x = \frac{(x_{2n} - x_0)}{2n}$ and $h_y = \frac{(y_{2m} - y_0)}{2m}$. The two dimensional integral therefore equates to:

$$I_{2D} = \int_{x_0}^{x_{2n}} \int_{y_0}^{y_{2m}} f(x, y) dy dx = \int_{x_0}^{x_{2n}} g(x) dx \quad (4.3.2)$$

With $g(x)$ defined as follows:

$$g(x) = \int_{y_0}^{y_{2m}} f(x, y) dy \approx \frac{h_y}{3} \left(f(x, y_0) + f(x, y_{2m}) + 4 \sum_{j=1}^m f(x, y_{2j-1}) + 2 \sum_{j=1}^{m-1} f(x, y_{2j}) \right)$$

Putting the approximation for $g(x)$ into equation (4.3.2) gives us:

$$\approx \int_{x_0}^{x_{2n}} \frac{h_y}{3} \left(f(x, y_0) + f(x, y_{2m}) + 4 \sum_{j=1}^m f(x, y_{2j-1}) + 2 \sum_{j=1}^{m-1} f(x, y_{2j}) \right) dx \quad (4.3.3)$$

By linearity of integral, this can be rewritten as four separate integrals:

$$= \frac{h_y}{3} \left(\int_{x_0}^{x_{2n}} f(x, y_0) dx + \int_{x_0}^{x_{2n}} f(x, y_{2m}) dx + 4 \sum_{j=1}^m \int_{x_0}^{x_{2n}} f(x, y_{2j-1}) dx + 2 \sum_{j=1}^{m-1} \int_{x_0}^{x_{2n}} f(x, y_{2j}) dx \right) \quad (4.3.4)$$

Using the composite Simpson's rule (4.3.1) on each of the four integrals yields the following:

$$\approx \frac{h_y}{3} \left(\begin{aligned} & \frac{h_x}{3} \left(f(x_0, y_0) + f(x_{2n}, y_0) + 4 \sum_{i=1}^n f(x_{2i-1}, y_0) + 2 \sum_{i=1}^{n-1} f(x_{2i}, y_0) \right) \\ & + \frac{h_x}{3} \left(f(x_0, y_{2m}) + f(x_{2n}, y_{2m}) + 4 \sum_{i=1}^n f(x_{2i-1}, y_{2m}) + 2 \sum_{i=1}^{n-1} f(x_{2i}, y_{2m}) \right) \\ & + 4 \sum_{j=1}^m \frac{h_x}{3} \left(f(x_0, y_{2j-1}) + f(x_{2n}, y_{2j-1}) + 4 \sum_{i=1}^n f(x_{2i-1}, y_{2j-1}) + 2 \sum_{i=1}^{n-1} f(x_{2i}, y_{2j-1}) \right) \\ & + 2 \sum_{j=1}^{m-1} \frac{h_x}{3} \left(f(x_0, y_{2j}) + f(x_{2n}, y_{2j}) + 4 \sum_{i=1}^n f(x_{2i-1}, y_{2j}) + 2 \sum_{i=1}^{n-1} f(x_{2i}, y_{2j}) \right) \end{aligned} \right) \quad (4.3.5)$$

As with the 2D composite trapezoid rule, we can simplify this expression. We can factorize $\frac{h_x}{3}$ out of the parenthesis. From there we first gather all the terms that are not in a sum $f(x_0, y_0)$, $f(x_{2n}, y_0)$, $f(x_0, y_{2m})$, and $f(x_{2n}, y_{2m})$. We can then gather the terms in the sums where one of the inputs in the function is fixed and independent of the sum. So we gather the terms where the y -coordinate is either fixed at y_0 or y_{2m} and the x -coordinate is indexed x_{2i-1} . We do the same in the terms where the x -coordinate is fixed at x_0 or x_{2n} and the y -coordinate is indexed y_{2j-1} . We can do the same for the sums where the x - and y -coordinates are indexed at x_{2i} and y_{2j} and the other coordinate is fixed. We then gather the double sums, first the sum where x is indexed x_{2i} and y is indexed y_{2j-1} and secondly the sum where x is indexed x_{2i-1} and y is indexed y_{2j} . Finally, we have a sum where x and y are indexed x_{2i} and y_{2j} and a sum where x and y are indexed x_{2i-1} and y_{2j-1} . This gives us the following:

$$= \frac{h_y h_x}{9} \left(\begin{aligned} & f(x_0, y_0) + f(x_{2n}, y_0) + f(x_0, y_{2m}) + f(x_{2n}, y_{2m}) \\ & + 4 \left(\sum_{i=1}^n [f(x_{2i-1}, y_0) + f(x_{2i-1}, y_{2m})] + \sum_{j=1}^m [f(x_0, y_{2j-1}) + f(x_{2n}, y_{2j-1})] \right) \\ & + 2 \left(\sum_{i=1}^{n-1} [f(x_{2i}, y_0) + f(x_{2i}, y_{2m})] + \sum_{j=1}^{m-1} [f(x_0, y_{2j}) + f(x_{2n}, y_{2j})] \right) \\ & + 8 \left(\sum_{j=1}^m \sum_{i=1}^{n-1} f(x_{2i}, y_{2j-1}) + \sum_{j=1}^{m-1} \sum_{i=1}^n f(x_{2i-1}, y_{2j}) \right) \\ & + 4 \sum_{j=1}^{m-1} \sum_{i=1}^{n-1} f(x_{2i}, y_{2j}) + 16 \sum_{j=1}^m \sum_{i=1}^n f(x_{2i-1}, y_{2j-1}) \end{aligned} \right) \quad (4.3.6)$$

We now have the formula for the 2D composite Simpson's rule. Next we want the computational complexity of the method. There are four terms that are not part of a sum and as such contribute constant work to the complexity regardless of n and m . Then there are two single sums, which each evaluate the function $2n$ and $2m$ times respectively. For the next two it is similar with $2(n-1)$ and $2(m-1)$ evaluations of the function. The four double sums each evaluate the function once in each of the iterations, meaning the total number of times they need to evaluate the function is $m(n-1)$, $(m-1)n$, $(m-1)(n-1)$, and mn . Summing up these terms and simplifying the expression gives the following computational complexity:

$$\begin{aligned} & 4 + 2n + 2m + 2(n-1) + 2(m-1) \\ & + m(n-1) + (m-1)n + (m-1)(n-1) + mn \\ & = 4nm + 2n + 2m + 1 \end{aligned}$$

This means that composite Simpson's rule in two dimensions has a computational complexity of $\mathcal{O}(nm)$. And if the number of steps in each dimension is equal such that $n = m = N$, the computational complexity is $\mathcal{O}(N^2)$.

4.3.1 Error of 2D composite Simpson's rule

We will derive the error term for the 2D composite Simpson's rule in a similar manner to the 2D composite midpoint and the trapezoid rules. As mentioned in section 2.3.3 the error term for the composite Simpson's rule is $-\frac{(b-a)h^4}{180}f^{(iv)}(c)$, $c \in [a, b]$. We will use this when applying the composite Simpson's rule to the 2D integral. The setting is the same as with the composite midpoint and composite trapezoid rules, now on the intervals $[x_0, x_{2n}]$ and $[y_0, y_{2m}]$:

$$I_{2D} = \int_{x_0}^{x_{2n}} \int_{y_0}^{y_{2m}} f(x, y) dy dx = \int_{x_0}^{x_{2n}} g(x) dx \quad (4.3.7)$$

With $g(x)$ defined as follows:

$$g(x) = \int_{y_0}^{y_{2m}} f(x, y) dy$$

We apply the composite Simpson's rule to $g(x)$. For the error term we take the fourth order partial derivative w.r.t. y :

$$g(x) = \frac{h_y}{3} \left(f(x, y_0) + f(x, y_{2m}) + 4 \sum_{j=1}^{m-1} f(x, y_{2j-1}) + 2 \sum_{j=1}^m f(x, y_{2j}) \right) - \frac{(y_{2m} - y_0)h_y^4}{180} \cdot \frac{\partial^4 f(x, c_{y_1})}{\partial y^4} \quad (4.3.8)$$

With $c_{y_1} \in [y_0, y_{2m}]$. Setting $g(x)$ to (4.3.8) in equation (4.3.7) yields:

$$\begin{aligned} &= \int_{x_0}^{x_{2n}} \frac{h_y}{3} \left(f(x, y_0) + f(x, y_{2m}) + 4 \sum_{j=1}^m f(x, y_{2j-1}) \right. \\ &\quad \left. + 2 \sum_{j=1}^{m-1} f(x, y_{2j}) \right) - \frac{(y_{2m} - y_0)h_y^4}{180} \cdot \frac{\partial^4 f(x, c_{y_1})}{\partial y^4} dx \end{aligned}$$

Factoring out $\frac{h_y}{3}$, applying the linearity of the integral and factoring out $-\frac{(y_{2m} - y_0)h_y^4}{180}$ from the error term gives us the following:

$$\begin{aligned} &\frac{h_y}{3} \left(\int_{x_0}^{x_{2n}} f(x, y_0) dx + \int_{x_0}^{x_{2n}} f(x, y_{2m}) dx + 4 \sum_{j=1}^m \int_{x_0}^{x_{2n}} f(x, y_{2j-1}) dx \right. \\ &\quad \left. + 2 \sum_{j=1}^{m-1} \int_{x_0}^{x_{2n}} f(x, y_{2j}) dx \right) - \frac{(y_{2m} - y_0)h_y^4}{180} \int_{x_0}^{x_{2n}} \frac{\partial^4 f(x, c_{y_1})}{\partial y^4} dx \end{aligned} \quad (4.3.9)$$

Now we apply the composite Simpson's rule to both the integrals in the parentheses and the integral in the error term. We include the error terms and isolate them so they come after the Simpson's rule approximations of the integral:

$$\begin{aligned}
 & \left(\frac{h_x}{3} \left(f(x_0, y_0) + f(x_{2n}, y_0) + 4 \sum_{i=1}^n f(x_{2i-1}, y_0) + 2 \sum_{i=1}^{n-1} f(x_{2i}, y_0) \right) \right. \\
 & + \frac{h_x}{3} \left(f(x_0, y_{2m}) + f(x_{2n}, y_{2m}) + 4 \sum_{i=1}^n f(x_{2i-1}, y_{2m}) + 2 \sum_{i=1}^{n-1} f(x_{2i}, y_{2m}) \right) \\
 & + 4 \sum_{j=1}^m \frac{h_x}{3} \left(f(x_0, y_{2j-1}) + f(x_{2n}, y_{2j-1}) + 4 \sum_{i=1}^n f(x_{2i-1}, y_{2j-1}) + 2 \sum_{i=1}^{n-1} f(x_{2i}, y_{2j-1}) \right) \\
 & + 2 \sum_{j=1}^{m-1} \frac{h_x}{3} \left(f(x_0, y_{2j}) + f(x_{2n}, y_{2j}) + 4 \sum_{i=1}^n f(x_{2i-1}, y_{2j}) + 2 \sum_{i=1}^{n-1} f(x_{2i}, y_{2j}) \right) \\
 & - \frac{(x_{2n} - x_0)h_x^4}{180} \cdot \frac{\partial^4 f(c_{x_1}, y_0)}{\partial x^4} - \frac{(x_{2n} - x_0)h_x^4}{180} \cdot \frac{\partial^4 f(c_{x_2}, y_{2m})}{\partial x^4} \\
 & - 4 \sum_{j=1}^m \frac{(x_{2n} - x_0)h_x^4}{180} \cdot \frac{\partial^4 f(c_{x_3}, y_{2j-1})}{\partial x^4} - 2 \sum_{j=1}^{m-1} \frac{(x_{2n} - x_0)h_x^4}{180} \cdot \frac{\partial^4 f(c_{x_4}, y_{2j})}{\partial x^4} \\
 & \left. - \frac{(y_{2m} - y_0)h_y^4}{180} \left(\frac{h_x}{3} \left(\frac{\partial^4 f(x_0, c_{y_1})}{\partial y^4} + \frac{\partial^4 f(x_{2n}, c_{y_1})}{\partial y^4} + 4 \sum_{i=1}^n \frac{\partial^4 f(x_{2i-1}, c_{y_1})}{\partial y^4} \right) \right. \right. \\
 & \left. \left. + 2 \sum_{i=1}^{n-1} \frac{\partial^4 f(x_{2i}, c_{y_1})}{\partial y^4} \right) - \frac{(x_{2n} - x_0)h_x^4}{180} \cdot \frac{\partial^8 f(c_{x_5}, c_{y_1})}{\partial y^4 \partial x^4} \right) \quad (4.3.10)
 \end{aligned}$$

With $c_{x_1}, c_{x_2}, c_{x_3}, c_{x_4}, c_{x_5} \in [x_0, x_{2n}]$. We can multiply $\frac{h_y}{3}$ on the terms in the big parentheses. That way, all error terms get multiplied by an extra step size. After multiplying $\frac{h_y}{3}$ on all terms, we can see that four of the terms correspond to the composite Simpson's rule (4.3.5). We can therefore ignore those terms, since they do not contribute to the error terms. Now we only look at the error terms from equation (4.3.10):

$$\begin{aligned}
 & - \frac{(x_{2n} - x_0)h_x^4 h_y}{540} \cdot \frac{\partial^4 f(c_{x_1}, y_0)}{\partial x^4} - \frac{(x_{2n} - x_0)h_x^4 h_y}{540} \cdot \frac{\partial^4 f(c_{x_2}, y_{2m})}{\partial x^4} \\
 & - \frac{(x_{2n} - x_0)h_x^4 h_y}{540} \left(4 \sum_{j=1}^m \frac{\partial^4 f(c_{x_3}, y_{2j-1})}{\partial x^4} \right) - \frac{(x_{2n} - x_0)h_x^4 h_y}{540} \left(2 \sum_{j=1}^{m-1} \frac{\partial^4 f(c_{x_4}, y_{2j})}{\partial x^4} \right) \\
 & - \frac{(y_{2m} - y_0)h_y^4 h_x}{540} \cdot \frac{\partial^4 f(x_0, c_{y_1})}{\partial y^4} - \frac{(y_{2m} - y_0)h_y^4 h_x}{540} \cdot \frac{\partial^4 f(x_{2n}, c_{y_1})}{\partial y^4} \\
 & - \frac{(y_{2m} - y_0)h_y^4 h_x}{540} \left(4 \sum_{i=1}^n \frac{\partial^4 f(x_{2i-1}, c_{y_1})}{\partial y^4} \right) - \frac{(y_{2m} - y_0)h_y^4 h_x}{540} \left(2 \sum_{i=1}^{n-1} \frac{\partial^4 f(x_{2i}, c_{y_1})}{\partial y^4} \right) \\
 & + \frac{(x_{2n} - x_0)(y_{2m} - y_0)h_x^4 h_y^4}{32400} \cdot \frac{\partial^8 f(c_{x_5}, c_{y_1})}{\partial y^4 \partial x^4} \quad (4.3.11)
 \end{aligned}$$

With the exception of the final error term, we will now put all terms with either $-\frac{(x_{2n}-x_0)h_x^4 h_y}{540}$ or $-\frac{(y_{2m}-y_0)h_y^4 h_x}{540}$ into two separate parentheses and factor out those constants:

$$\begin{aligned}
 & - \frac{(x_{2n} - x_0)h_x^4 h_y}{540} \left(\frac{\partial^4 f(c_{x_1}, y_0)}{\partial x^4} + \frac{\partial^4 f(c_{x_2}, y_{2m})}{\partial x^4} + 4 \sum_{j=1}^m \frac{\partial^4 f(c_{x_3}, y_{2j-1})}{\partial x^4} + 2 \sum_{j=1}^{m-1} \frac{\partial^4 f(c_{x_4}, y_{2j})}{\partial x^4} \right) \\
 & - \frac{(y_{2m} - y_0)h_y^4 h_x}{540} \left(\frac{\partial^4 f(x_0, c_{y_1})}{\partial y^4} + \frac{\partial^4 f(x_{2n}, c_{y_1})}{\partial y^4} + 4 \sum_{i=1}^n \frac{\partial^4 f(x_{2i-1}, c_{y_1})}{\partial y^4} + 2 \sum_{i=1}^{n-1} \frac{\partial^4 f(x_{2i}, c_{y_1})}{\partial y^4} \right) \quad (4.3.12) \\
 & + \frac{(x_{2n} - x_0)(y_{2m} - y_0)h_x^4 h_y^4}{32400} \cdot \frac{\partial^8 f(c_{x_5}, c_{y_1})}{\partial y^4 \partial x^4}
 \end{aligned}$$

Similar to what we did with the error terms for the composite midpoint rule and the composite trapezoid rule, we will now use theorem (2.2.1) to rewrite the sum of partial derivatives as a constant times the partial derivative at a certain point in the interval. In each of the two parentheses in equation (4.3.12) we have two terms that are multiplied by 1 and two sums, one with m or n terms and one with $m - 1$ or $n - 1$ terms. These last two are multiplied by 4 and 2 respectively, so the sum of constants for each parentheses will be $1 + 1 + 4m + 2(m - 1) = 6m$ and $1 + 1 + 4n + 2(n - 1) = 6n$ respectively. Using theorem (2.2.1), the error terms can be rewritten as follows:

$$\begin{aligned}
 & = - \frac{(x_{2n} - x_0)h_x^4 h_y}{540} \left(6m \frac{\partial^4 f(c_{x_5}, c_{y_2})}{\partial x^4} \right) - \frac{(y_{2m} - y_0)h_y^4 h_x}{540} \left(6n \frac{\partial^4 f(c_{x_6}, c_{y_1})}{\partial x^4} \right) \quad (4.3.13) \\
 & + \frac{(x_{2n} - x_0)(y_{2m} - y_0)h_x^4 h_y^4}{32400} \cdot \frac{\partial^8 f(c_{x_7}, c_{y_1})}{\partial y^4 \partial x^4}
 \end{aligned}$$

With $c_{x_5}, c_{x_6}, c_{x_7} \in [x_0, x_{2n}]$ and $c_{y_2} \in [y_0, y_{2m}]$. For the composite Simpson's rule, the relations between h_x, h_y and n, m are $n = \frac{(x_{2n} - x_0)}{2h_x}$ and $m = \frac{(y_{2m} - y_0)}{2h_y}$ respectively. Substituting n and m in equation (4.3.13) allows us to cancel the h_y and h_x in the first and second terms respectively. We then get the following error terms:

$$\begin{aligned}
 & = - \frac{(x_{2n} - x_0)(y_{2m} - y_0)h_x^4}{180} \cdot \frac{\partial^4 f(c_{x_5}, c_{y_2})}{\partial x^4} - \frac{(x_{2n} - x_0)(y_{2m} - y_0)h_y^4}{180} \cdot \frac{\partial^4 f(c_{x_6}, c_{y_1})}{\partial y^4} \quad (4.3.14) \\
 & + \frac{(x_{2n} - x_0)(y_{2m} - y_0)h_x^4 h_y^4}{32400} \cdot \frac{\partial^8 f(c_{x_7}, c_{y_1})}{\partial y^4 \partial x^4}
 \end{aligned}$$

The first two terms feature h_x^4 and h_y^4 respectively while the final term features $h_x^4 h_y^4$. As with the error terms for the composite midpoint and trapezoid rules, the lower power terms decay at a slower rate as h_x and h_y go to 0. The order of accuracy of the 2D composite Simpson's rule is $\mathcal{O}(h_x^4 + h_y^4)$ because of the two terms involving h_x^4 and h_y^4 . As we did for the composite trapezoid and midpoint rules, we need the order of the error to be based on the number of steps. This means we need to substitute $h_x^4 = \frac{(x_n - x_0)^4}{2^4 n^4}$ and $h_y^4 = \frac{(y_n - y_0)^4}{2^4 m^4}$ into equation (4.3.14) to get:

$$\begin{aligned}
 & - \frac{(x_{2n} - x_0)^5 (y_{2m} - y_0)}{2880 n^4} \cdot \frac{\partial^4 f(c_{x_5}, c_{y_2})}{\partial x^4} - \frac{(x_{2n} - x_0)(y_{2m} - y_0)^5}{2880 m^4} \cdot \frac{\partial^4 f(c_{x_6}, c_{y_1})}{\partial y^4} \\
 & + \frac{(x_{2n} - x_0)^5 (y_{2m} - y_0)^5}{8294400 n^4 m^4} \cdot \frac{\partial^8 f(c_{x_7}, c_{y_1})}{\partial y^4 \partial x^4}
 \end{aligned}$$

Then as argued for the order based on step size, the first two error terms will decay at a slower rate than the final error term as n and m grow. This means the order of accuracy of the 2D composite Simpson's rule is $\mathcal{O}(n^{-4} + m^{-4})$. And if the number of steps used in each dimension is equal such that $n = m = N$, the order of accuracy of the method is $\mathcal{O}(N^{-4})$.

4.4 Computational complexity of higher dimensional Newton-Cotes methods

As we have seen in the previous sections, the computational complexity of Newton-Cotes methods in one dimension is $\mathcal{O}(N)$ as noted in section 2.3. And in sections 4.1, 4.2, and 4.3 we have shown that the computational complexity for the 2D extensions of the Newton-Cotes methods are all $\mathcal{O}(N^2)$ if the number of steps is the same for both dimensions.

It therefore stands to reason that if we tried to derive Newton-Cotes methods for higher dimensions, the computational complexity would grow at a similar rate. In particular, if the number of steps in all dimensions is the same N , then the computational complexity of Newton-Cotes methods on a d -dimensional function would be $\mathcal{O}(N^d)$. This means that the computational complexity of Newton-Cotes methods grows exponentially as a function of the dimension d .

We now have all the necessary methods and background results needed to perform the numerical integration of the function.

5 Numerical experiments

We are now ready to numerically approximate the integral of our function. For the following experiments we will look at the integral on the interval $[-3, 3]$. As noted in section 3.1, our function is symmetric. We can therefore also approximate the true integral on the interval $[0, B]$ and multiply the result by 2 to approximate the integral on the interval $[-B, B]$. In addition to the bounds of integration, we also want to assess the numerical accuracy of the previously discussed methods for different numbers of steps for Newton-Cotes and points for Monte Carlo methods respectively. For all proceeding methods, we will approximate the integral of the function with values of $\lambda = 0, 1, \dots, 8$. The object of interest is the error and its relation to λ for the different methods. To that end, we will also evaluate the relative error. The relative error is $\frac{I_t - I_a}{I_t}$, where I_t is the true solution of the integral on the given interval and I_a is our approximation of the integral on the given interval. The absolute relative error is $\left| \frac{I_t - I_a}{I_t} \right|$. We will use the relative error to study of rate of decay of the error for each method and to compare the accuracy of the different methods.

We will also be looking at the integral on the two-dimensional function $e^{-(x_1+x_2)^2} \cos(\lambda_1 x_1 + \lambda_2 x_2)$ on the domain $[-3, 3] \times [-3, 3]$. We will consider uniform λ in two dimensions, that is $\lambda_1 = \lambda_2$. In both one and two dimensions we will use the previously discussed Newton-Cotes methods and Monte Carlo integration methods. The number of steps/points we use for Newton-Cotes and Monte Carlo in one dimension and for Monte Carlo in two dimensions will be $N = 2^i$, $i = 4, \dots, 29$, while for Newton-Cotes in two dimensions we use $N = 2^i$, $i = 4, \dots, 14$ number of steps. The reason we use fewer points for Newton-Cotes in two dimensions is because of the higher computational complexity compared to the one-dimensional case, as discussed in section 4.4. The numerical integrations were performed with MATLAB using the files the appended zip file `BAMM501_code_files.zip`. All the data is also contained in the zip file.

5.1 One-dimensional Newton-Cotes methods

First, we look at Newton-Cotes in one dimension. We use the composite midpoint, trapezoid, and Simpson's rules with the previously mentioned different numbers of points on the interval $[-3, 3]$. For each of the three methods we also evaluate them on the interval $[0, 3]$. When we do so, we halve the number of steps and multiply the results by 2. We halve the number of steps so the step size is equal compared the whole interval.

Since we use three methods, each on two different intervals, we effectively use six Newton-Cotes methods to numerically approximate the integral of $e^{-x^2} \cos(\lambda x)$. Once we have the approximations of the integral for each number of steps and each λ , we then compute the relative error of our approximations compared to the true solution of the integral on the given interval. This means that for the methods evaluated on the interval $[-3, 3]$ we compare the approximation with the true integral on the interval $[-3, 3]$. And for the methods used on the interval $[0, 3]$ we compared the results with the integral over the interval $[0, 3]$ multiplied by 2. That is to say, we compare the results evaluated over both intervals with the true solution, (3.2.1). The numerical integration was performed by file `NewtonCotesIntegrationOneDim.m`.

Once we have the relative errors for each number of steps and each λ , we then plot the $\log_2$ of the number of steps and the $\log_2$ of the absolute value of the relative error. We then create plots for four different values of λ . The resulting plots for values of $\lambda = 1, 3, 6$, and 8 can be seen in figure 3.

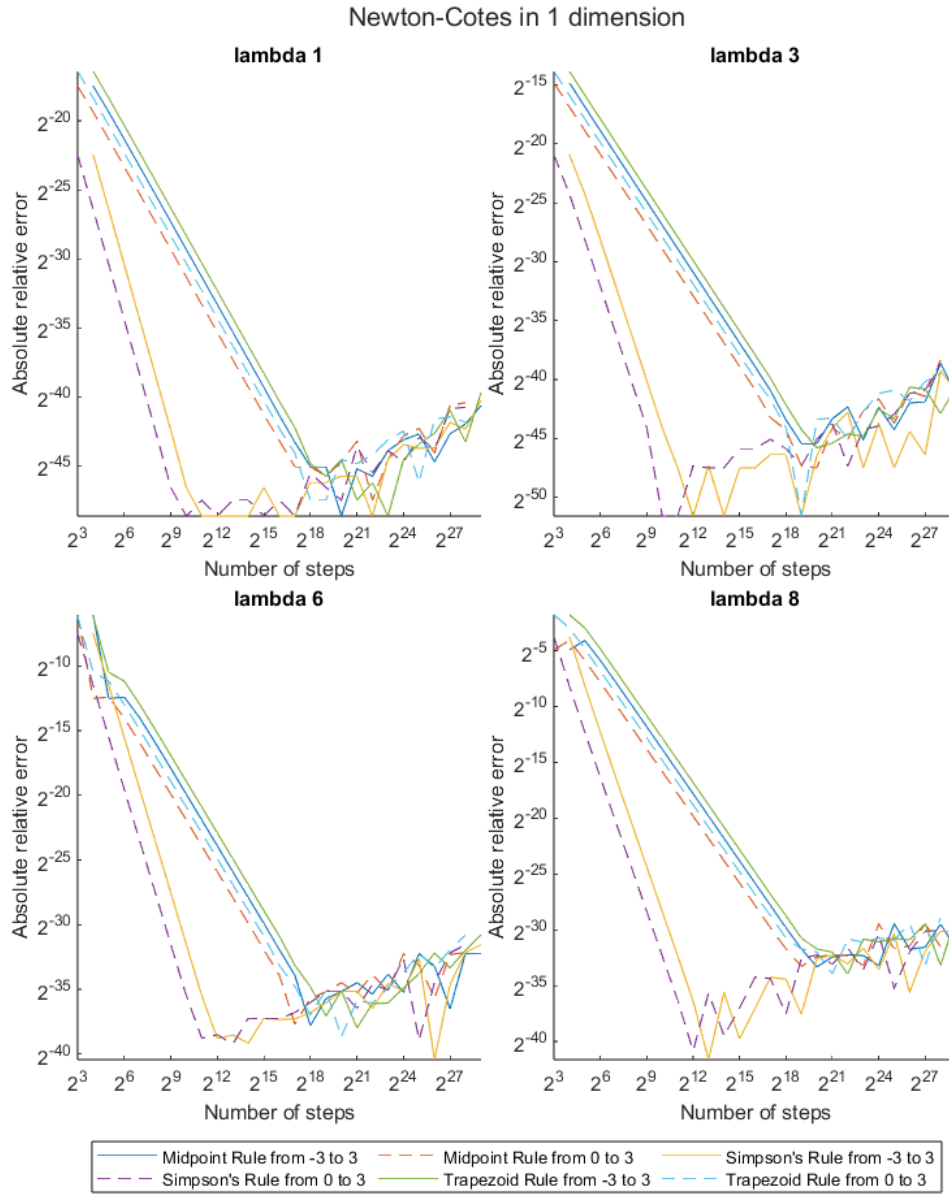


Figure 3: The absolute relative error is plotted on the vertical axis against the number of steps plotted on the horizontal axis. The upper left plot is the error for $\lambda = 1$, the upper right plot is the error for $\lambda = 3$, the bottom left plot is the error for $\lambda = 6$, and the bottom right plot is the error for $\lambda = 8$. Made with NCDatPlot.m.

As we can see, the plots for different values of λ look similar in terms of how absolute relative error behaves as the number of steps increases. The difference λ makes is the magnitude of the error. For $\lambda = 1$ the range of the vertical axis is between roughly 2^{-15} and 2^{-50} , while the range changes for larger values of λ . When $\lambda = 8$, the range of the vertical axis is between roughly 2^0 and 2^{-45} as seen in figure 3.

As for the plots themselves, we can see that the error decline behaves linearly for all methods until a certain number of steps. For composite Simpson's rule, the error declines linearly until about 2^{11} number of steps after which the error begins to grow and oscillate for higher numbers of steps. This is for composite Simpson's rule applied to the interval $[-3, 3]$. When the composite Simpson's rule is applied on the interval $[0, 3]$, the error begins to oscillate at 2^{10} number of steps. For the composite trapezoid and composite midpoint rules, the error declines linearly until about 2^{19} after which the error begins to grow and oscillate. This is the case when the methods are applied to $[-3, 3]$. When applied to the interval $[0, 3]$, the error begins to oscillate at 2^{18} for the composite midpoint and trapezoid rules. The error begins at more or less the same number of steps for all methods regardless of the value of λ .

The reason for this oscillation is partly because of machine precision. When the number of steps is large, the integral sum becomes larger the further along the interval we are at. Since the sum is of large magnitude, the function values that are added need to be scaled appropriately because of machine precision and as a result lose significant digits.

As for why the composite Simpson's rule begins to oscillate at a smaller number of steps compared to the composite midpoint and trapezoid rules, it is partly because it gets as close to accurately approximating the true solution of the integral as machine precision allows. It therefore runs into the problem with machine precision earlier than the other methods.

Since the error decline is linear up to a certain number of steps for all methods shown in figure 3, we want to investigate the slope of the log absolute relative error via linear regression. We do this for the numbers of steps before the error of the methods begins to oscillate. The reason for the log is that the error decay of a given Newton-Cotes method is proportional to $\frac{1}{N^p}$, where N is the number of steps and p is the order of the given method as discussed in section 2.3. If we take the log of the error, this gives us:

$$\log\left(\frac{1}{N^p}\right) = \log(1) - p \log(N) = -p \log(N)$$

We can therefore estimate the rate of decay of the error via the slope coefficient of a first degree polynomial fitted to the error. The results can be seen in table 1.

lambda	1	3	6	8
Midpoint rule from -3 to 3	-1.989	-2.017	-2.028	-1.915
Midpoint rule from 0 to 3	-1.99	-2.002	-1.979	-1.903
Simpson's rule from -3 to 3	-4.011	-3.896	-4.004	-4.082
Simpson's rule from 0 to 3	-3.992	-3.863	-4.006	-4.106
Trapezoid rule from -3 to 3	-2.009	-2.008	-1.98	-1.97
Trapezoid rule from 0 to 3	-1.991	-1.997	-1.98	-1.97

Table 1: A table where each row corresponds to a method and each column corresponds to a value of λ . Each entry is the slope coefficient derived via linear regression for the given method and value of λ . Made with `NCErrorOrder.m`.

As we can see in table 1, the slope coefficient for the log absolute relative error of the composite Simpson's rule is around -4 and the slope coefficient for the error of the composite trapezoid and midpoint rules are both around -2 . This matches our expectations from sections 2.3.1, 2.3.2, and 2.3.3, that the error decay for the different methods are order 4 for composite Simpson's rule and 2 for the composite trapezoid and midpoint rules.

5.2 One-dimensional Monte Carlo integration methods

We then look at methods of one-dimensional Monte Carlo integration. As previously discussed, we will use both simple sampling and importance sampling. When using simple sampling, we will use a uniform distribution on the interval $[-3, 3]$. We will also use the symmetry of the function and sample using a uniform distribution on the interval $[0, 3]$ and multiply the results by 2.

For importance sampling we will use a standard normal distribution and a half-normal distribution. The reason we use these distributions is because the Gaussian exponential function is part of the function we want to integrate. The support for the standard normal distribution is the real line $\mathbb{R}$ and the support for the half-normal distribution is the interval $[0, \infty)$. We will therefore compare the integral results for importance sampling using the standard normal distribution with the true solution of the integral over the real line, (3.3.1). When we sample with the half-normal distribution, we also compare the integral results with the true integral on the real line, (3.3.1). As we did in section 5.1, we evaluate the methods over the intervals $[0, 3]$ and $[0, \infty)$ with $N = 2^i$, $i = 3, \dots, 28$ number of points instead of $N = 2^i$, $i = 4, \dots, 29$ number of points, as we do on the intervals $[-3, 3]$ and $(-\infty, \infty)$.

As with Newton-Cotes in one dimension, we want to approximate the true solution of the integral and compare the relative errors. For Monte Carlo however, because of the inherent randomness of sampling, we run the numerical integration multiple times using different seeds. For each seed we approximate the integral for all combinations of numbers of points and values of λ . The numerical integration was performed by file `MonteCarloIntegrationOneDim.m`. Once we have the integral results, we then compute the absolute relative error for each of these results. Finally, we take the mean of the $\log_2$ of the absolute relative error across the different seeds and plot this in a error bar plot which can be seen in figure 4.

As was case for the errors for the Newton-Cotes methods shown in figure 3, the value of λ affects the magnitude of the error as shown in figure 4. For $\lambda = 1$, the range of the vertical axis is roughly between 2^0 and 2^{-20} , while for $\lambda = 8$, the range of the vertical axis is roughly between 2^{23} and 2^0 . The way the random numbers were generated means that the plots for the error using the normal and half-normal distributions overlap. What happens is that a normally distributed random number is generated and then the absolute value is taken to simulate the half-normal distribution. Since the function we are trying to integrate is symmetrical, the function evaluated at the given random number will then be the same regardless of the sign of the number, i.e $f(-x) = f(x)$, $\forall x \in \mathbb{R}$.

As mentioned earlier, the methods that are applied to the intervals $[0, 3]$ and $[0, \infty)$ are evaluated with 2^i , $i = 3, \dots, 28$ number of points. That is why the lines that correspond to importance sampling using the half-normal distribution and simple sampling on the interval $[0, 3]$ start at 2^3 number of points and end at 2^{28} number of points in figure 4. As for the actual plots, we can see that for $\lambda = 1$, simple and importance sampling have a similar magnitude of error, with importance sampling generally having a smaller error. For $\lambda = 3$ and $\lambda = 6$ the errors for the two methods are closer to each other. For $\lambda = 8$ however the error for importance samplings is larger than the error for simple sampling for every different number of points. The reason for this can be that importance sampling works well when the weight function has a degree of correlation with the function, we wish to approximate the integral of. This is the case for $\lambda = 1$ since the function $e^{-x^2} \cos(\lambda x)$ still broadly behaves similarly to a normal density function with expectation 0. When λ is larger however, the function behaves less and less like the normal density function and as such is expected to a smaller degree of correlation. In such a situation it is generally more efficient to use simple sampling.

We will now investigate the rate of decay of the log absolute error. For Monte Carlo integration, the error is proportional to $\frac{1}{\sqrt{N}}$, where N is the sample size as shown in section 2.4. If we take the log of this expression:

$$\log\left(\frac{1}{\sqrt{N}}\right) = \log(1) - \frac{1}{2} \log(N) = -\frac{1}{2} \log(N)$$

The table for the slope coefficients for the first degree polynomials fitted to the log absolute relative errors for the Monte Carlo integration methods can be seen in table 2. We can see in table 2, the log of the absolute relative error declines linearly with coefficients around $-\frac{1}{2}$ for all methods. This means that the actual error decay is roughly proportional to $\frac{1}{\sqrt{N}}$ as expected.

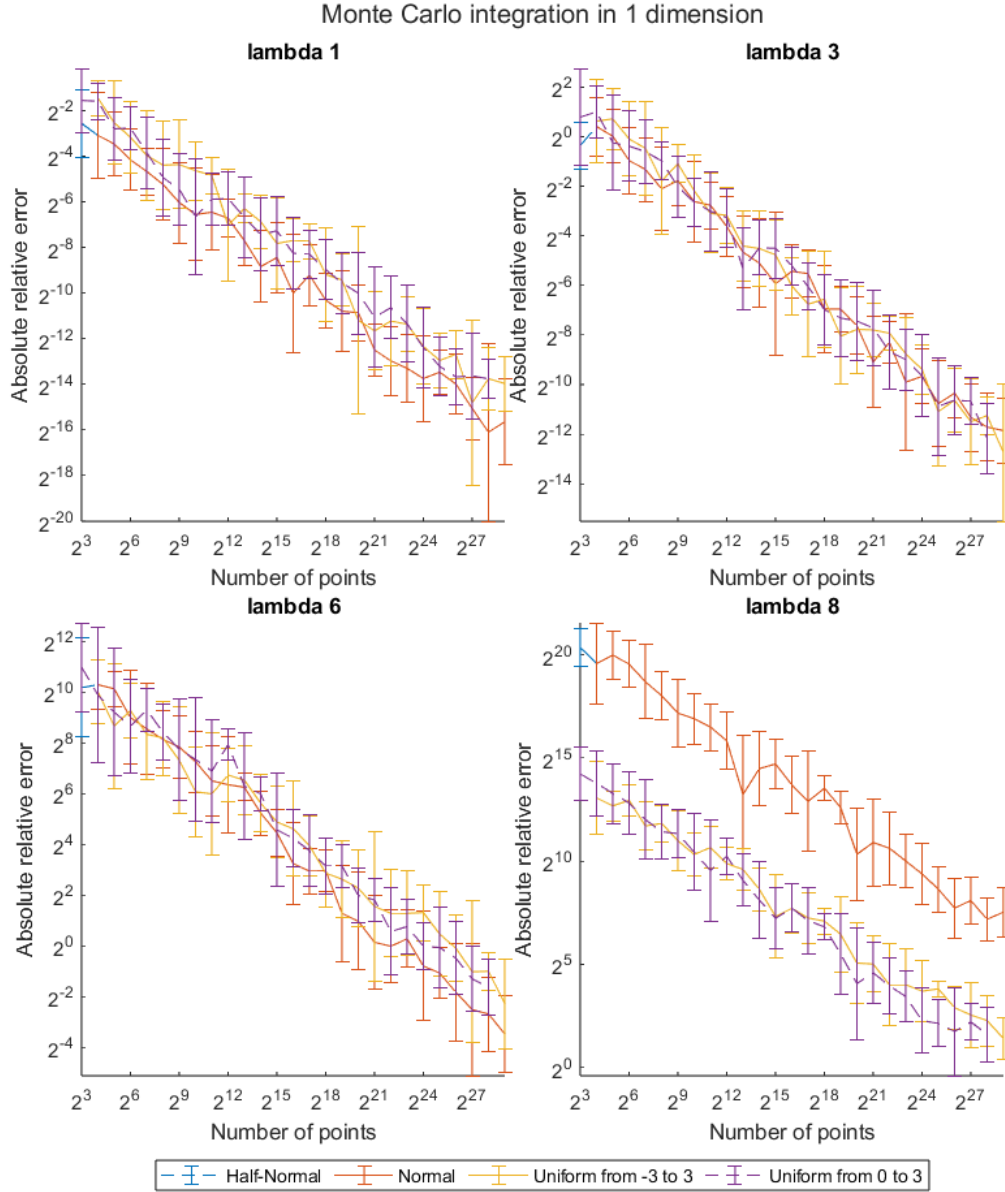


Figure 4: The mean of the log of the absolute error is plotted on the vertical axis against the number of points on the horizontal axis. The standard deviation of the error is plotted as error bars. The upper left plot is the error for $\lambda = 1$, the upper right plot is the error for $\lambda = 3$, the bottom left plot is the error for $\lambda = 6$, and the bottom right plot is the error for $\lambda = 8$. Made with `MCDataplot.m`.

lambda	1	3	6	8
Half-normal	-0.522	-0.496	-0.556	-0.537
Normal	-0.518	-0.502	-0.562	-0.531
Uniform from -3 to 3	-0.512	-0.531	-0.458	-0.48
Uniform from 0 to 3	-0.494	-0.516	-0.502	-0.536

Table 2: A table where each row corresponds to a method and each column corresponds to a value of λ . Each entry is the slope coefficient derived via linear regression for the distribution used for sampling and value of λ . Made with `MCErrOrder.m`.

5.3 Error comparison between Newton-Cotes and Monte Carlo methods in one dimension

Since we want to see which methods are more efficient for integrating the given function, we can compare the error for the different number of steps/points for both the Newton-Cotes and Monte Carlo methods in one dimension. Graphs comparing the methods for specific values of λ can be seen in figure 5.

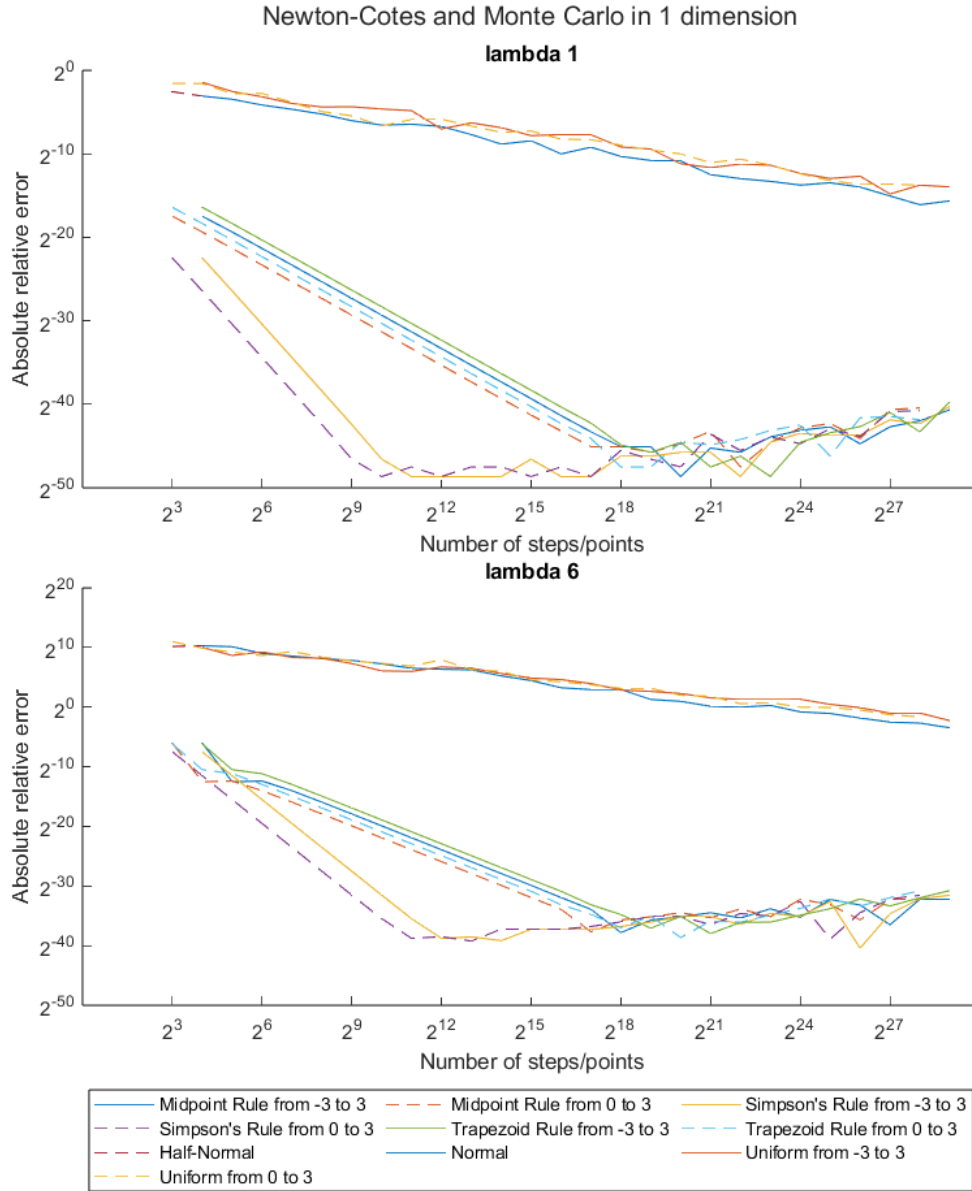


Figure 5: The upper plot is the log absolute relative error of the Newton-Cotes and Monte Carlo integration methods when $\lambda = 1$ and the lower plot is for $\lambda = 6$. Made with `NCvsMCDataPlot.m`.

In the plot for $\lambda = 1$ we can see that the error for Monte Carlo ranges between roughly 2^0 and 2^{-10} while the error for the Newton-Cotes methods ranges between roughly 2^{-15} and 2^{-50} . In the plot for $\lambda = 6$, the error for Monte Carlo ranges between roughly 2^{10} and 2^0 , while the error for the Newton-Cotes methods ranges between roughly 2^{-10} and 2^{-40} .

As we can see, the magnitude of the error is larger for Monte Carlo and the rate of decay of the error is faster for Newton-Cotes as previously discussed. This shows that in one dimension, the Newton-Cotes methods are generally more reliable in accurately approximating the true solution of the integral. The value of λ seems to affect the magnitude of the error to a similar degree for both the Monte Carlo and Newton-Cotes methods.

Since the computational complexity for both Newton-Cotes and Monte Carlo is of order $\mathcal{O}(N)$ as shown in sections 2.3 and 2.4, there is no noticeable advantage in using Monte Carlo compared to Newton-Cotes since the error is generally smaller for Newton-Cotes.

5.4 Two-dimensional Newton-Cotes methods

Now for the numerical integration of the two-dimensional function using Newton-Cotes methods. As mentioned earlier, for two-dimensional Newton-Cotes we use $N = 2^i$, $i = 4, \dots, 14$ number of steps. This is because the computational complexity for Newton-Cotes methods grows exponentially as a function of the dimension as noted in section 4.4. Since we are looking at the two-dimensional case, 2^{14} steps in both dimensions is equal to 2^{28} total number of steps. For the two-dimensional case we will be using the 2D composite midpoint, trapezoid and Simpson's rules derived earlier. We again approximate the integral using each method for each number of steps and each value of λ . The numerical integration was performed by file `NewtonCotesIntegrationTwoDim.m`. The plot for the log absolute relative error for each method is shown in figure 6.

As was the case in section 5.1, the value of λ only affects the magnitude of the error and not the rate of decay of the error. The range of the vertical axis for $\lambda = 1$ is roughly between 2^{-15} and 2^{-45} , while the range of the vertical axis for $\lambda = 8$ is roughly between 2^0 and 2^{-30} . As could be seen for the one-dimensional Newton-Cotes methods in figure 3, the rate of decay of the error is mostly linear until a certain number of steps. For the composite Simpson's rule, the error decays linearly until around 2^8 to 2^{11} number of steps for the different values of λ . Afterwards, the error begins to oscillate as it did for the composite Simpson's rule in one dimension. As with the one-dimensional case, this oscillation can be attributed to machine precision for the same reasons as discussed in section 5.1. For the composite trapezoid and midpoint rules, the error decays linearly for all the different numbers of steps in two dimensions. This can be because the number of steps required for the two methods to begin to oscillate in section 5.1 was around 2^{18} and we do not use as many points for the two-dimensional Newton-Cotes methods.

As in section 5.1, we want to estimate the rate of decay of the error for the methods. The slope coefficients of the first degree polynomials fitted to the log of the errors of the 2D composite midpoint and 2D composite trapezoid rules should be around -2 as discussed in sections 4.1.1, and 4.2.1. And the slope coefficient for the first degree polynomial fitted to the log of the error of the 2D composite Simpson's rule should be around -4 as discussed in section 4.3.1. The table of slope coefficients of the polynomials fitted to the errors for the different methods is shown in table 3.

As can be seen in table 3, the slope coefficients are around -2 for the 2D composite midpoint and trapezoid rules and around -4 for the 2D composite Simpson's rule. This means that the error decay for the 2D composite midpoint and trapezoid rules is of order 2 while the error decay for the 2D composite Simpson's rule is of order 4. This matches our expectations from the derivations of error terms in sections 4.1.1, 4.2.1, and 4.3.1.

lambda	1	3	6	8
Midpoint rule	-1.989	-2.005	-1.975	-1.856
Simpson's rule	-3.844	-3.845	-4.119	-4.186
Trapezoid rule	-1.998	-2.003	-1.97	-1.934

Table 3: A table where each row corresponds to a method and each column corresponds to a value of λ . Each entry is the slope coefficient derived via linear regression for the given method and value of λ . Made with `NCErrorOrder.m`.

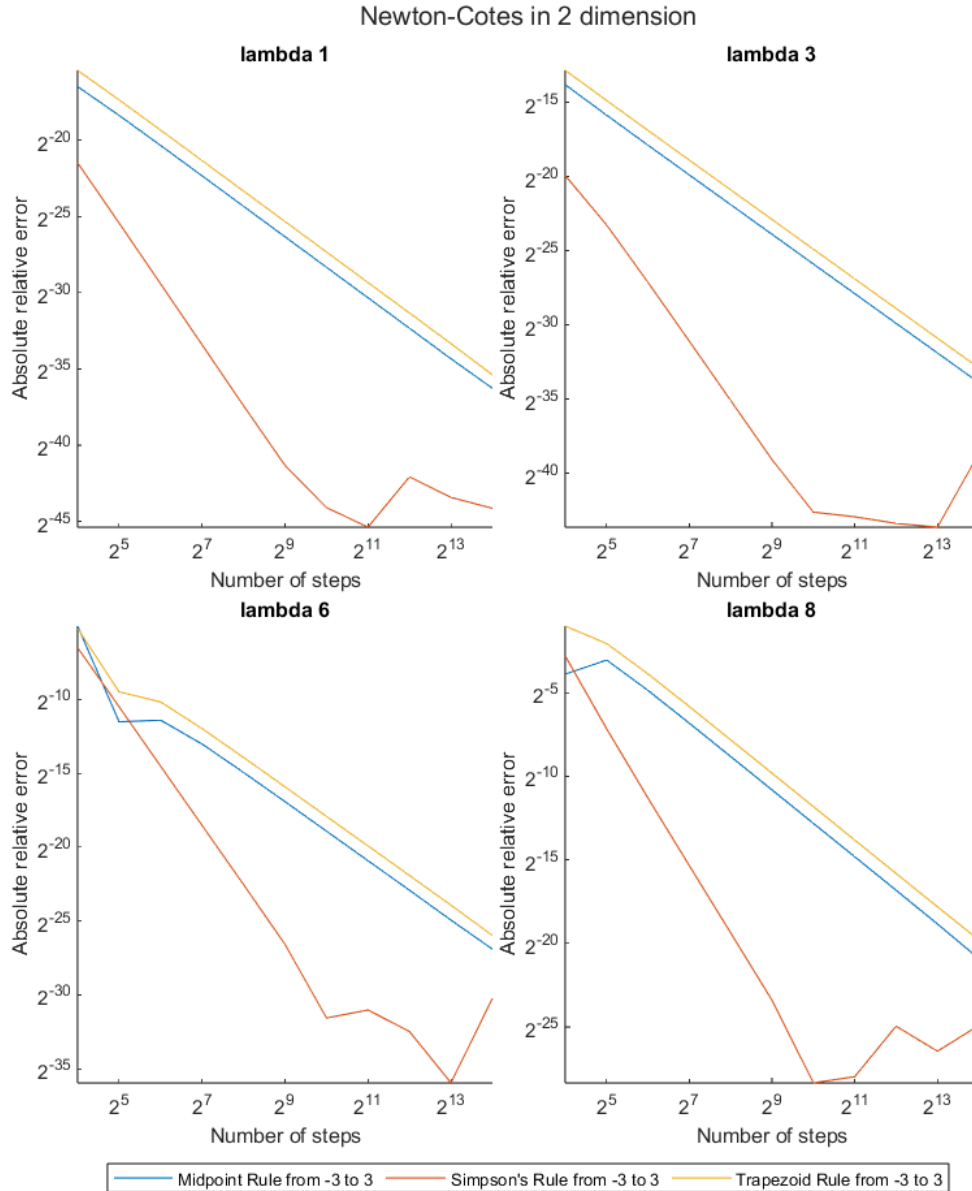


Figure 6: The log of the absolute relative error is plotted on the vertical axis against the number of steps plotted on the horizontal axis. The upper left plot is the error for $\lambda = 1$, the upper right plot is the error for $\lambda = 3$, the bottom left plot is the error for $\lambda = 6$, and the bottom right plot is the error for $\lambda = 8$. Made with NCDatPlot.m.

5.5 Two-dimensional Monte Carlo integration methods

Now we look at Monte Carlo integration in two dimensions. As in the one-dimensional case in section 5.2 we use simple and importance sampling. Simple sampling in the two-dimensional case is analogous to using a bivariate uniform distribution on the domain $\Omega = [-3, 3]^2$. For importance sampling, we sample using a bivariate standard normal distribution, that is to say the expectation μ is equal to 0 for both normal distributions. And the covariance matrix Σ is equal to the identity matrix. As in the one-dimensional case, we sample using $N = 2^i$, $i = 4, \dots, 29$ number of points. And as in the one-dimensional case, we will run several Monte Carlo simulations with different seeds and then evaluate the mean of the error for the different seeds. The numerical integration was performed by file `MonteCarloIntegrationMultiDim.m`. Once we have

our integral approximations and the absolute relative error, we take the $\log_2$ of the errors and plot the mean errors with error bars against the different number of points. The results can be seen in figure 7.

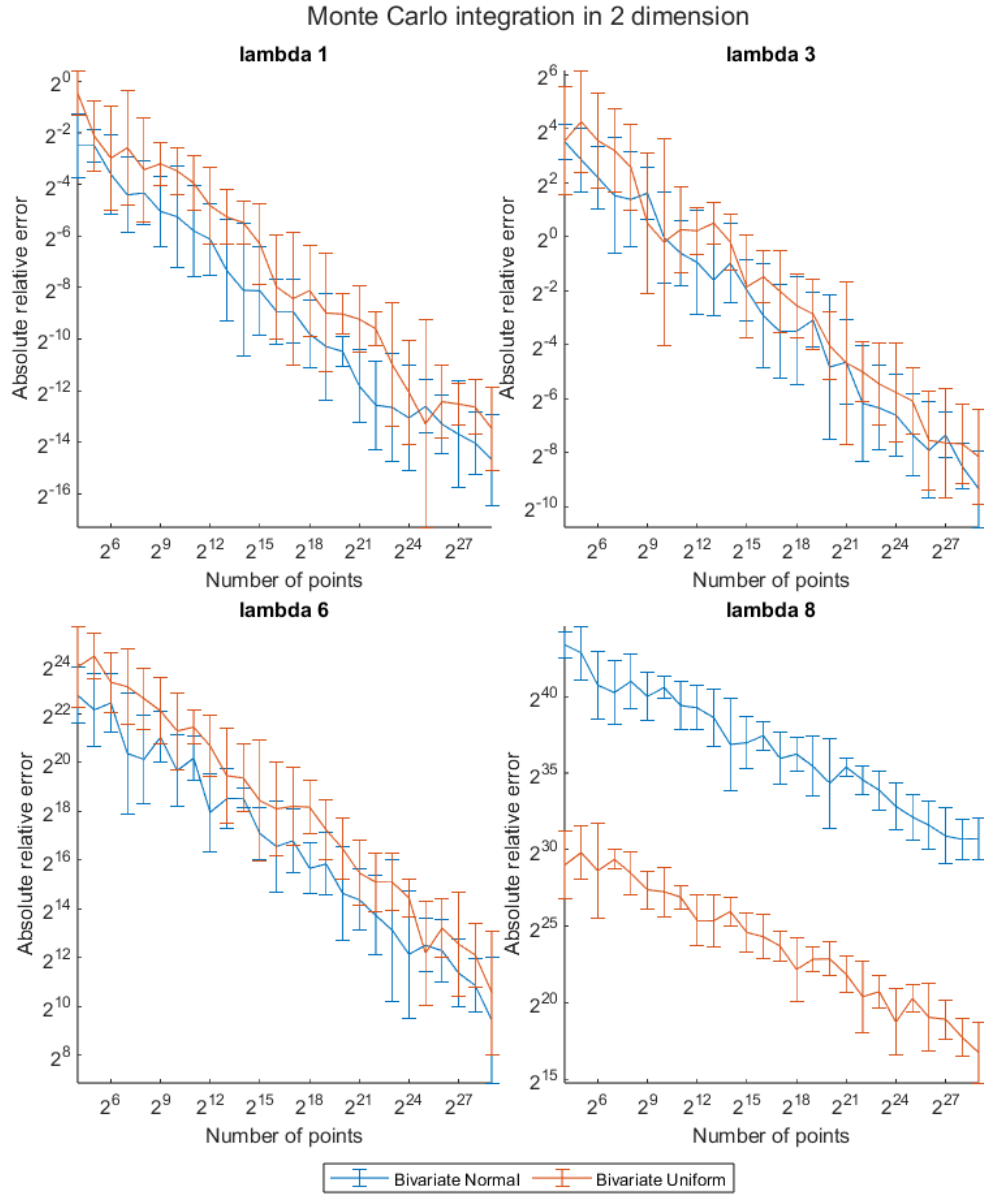


Figure 7: The mean of the log of the absolute error is plotted on the vertical axis against the number of points on the horizontal axis. The standard deviation of the error is plotted as error bars. The upper left plot is the error for $\lambda = 1$, the upper right plot is the error for $\lambda = 3$, the bottom left plot is the error for $\lambda = 6$, and the bottom right plot is the error for $\lambda = 8$. Made with `MCDatPlot.m`.

As was the case with the results for one-dimensional Monte Carlo integration methods in figure 4, the value of λ only affects the magnitude of the error. Another thing we see in figure 7 is that, just as in figure 4, the error becomes larger for importance sampling compared to simple sampling when $\lambda = 8$, while the error for importance sampling is smaller compared to simple sampling when $\lambda = 1$. The reason for this is similar to the case in one-dimension, where the multivariate normal probability density function is expected to have a smaller correlation with the function for larger values of λ . In such a situation, simple sampling is more efficient for accurately approximating the integral of our function.

Now for the error decay. As in section 5.2, the error decay for two dimensional Monte Carlo integration is proportional to $\frac{1}{\sqrt{N}}$, where N is the sample size as discussed in section 2.4. As in the one-dimensional case, we expect to see slope coefficients around $-\frac{1}{2}$. The table for the slope coefficients of the polynomials fitted to the errors can be seen in table 4.

lambda	1	3	6	8
Bivariate normal	-0.503	-0.495	-0.504	-0.494
Bivariate uniform	-0.514	-0.501	-0.53	-0.507

Table 4: A table where each row corresponds to a method and each column corresponds to a value of λ . Each entry is the slope coefficient derived via linear regression for the distribution used for sampling and value of λ . Made with `MCErrorOrder.m`.

For both the bivariate uniform and normal distributions the slope coefficients are around $-\frac{1}{2}$ for all values of λ as expected.

5.6 Error comparison between Newton-Cotes and Monte Carlo methods in two dimensions

As we did in section 5.3, we want to compare the errors between the Newton-Cotes methods and the Monte Carlo integration methods in two dimensions. The plots for the errors are shown in figure 8.

When $\lambda = 1$, the magnitude of the error for Monte Carlo ranges between roughly 2^0 and 2^{-5} and for Newton-Cotes the error ranges between roughly 2^{-15} and 2^{-40} . When $\lambda = 6$, the magnitude of the error for Monte Carlo ranges between roughly 2^{25} and 2^{20} , while for Newton-Cotes the error ranges between roughly 2^{-5} and 2^{-35} .

As was the case in section 5.3, the error is larger for the Monte Carlo integration methods than for the Newton-Cotes methods for the different numbers of points/steps and the error decays at a slower rate. The complexity of Newton-Cotes grows exponentially as a function of the dimension while the computational complexity of Monte Carlo integration grows linearly as a function of the dimension. Therefore, while Newton-Cotes methods still produce more accurate results in two dimensions, as they do in the one-dimensional case, once the dimension becomes too large Monte Carlo integration methods are expected to become more computationally efficient.

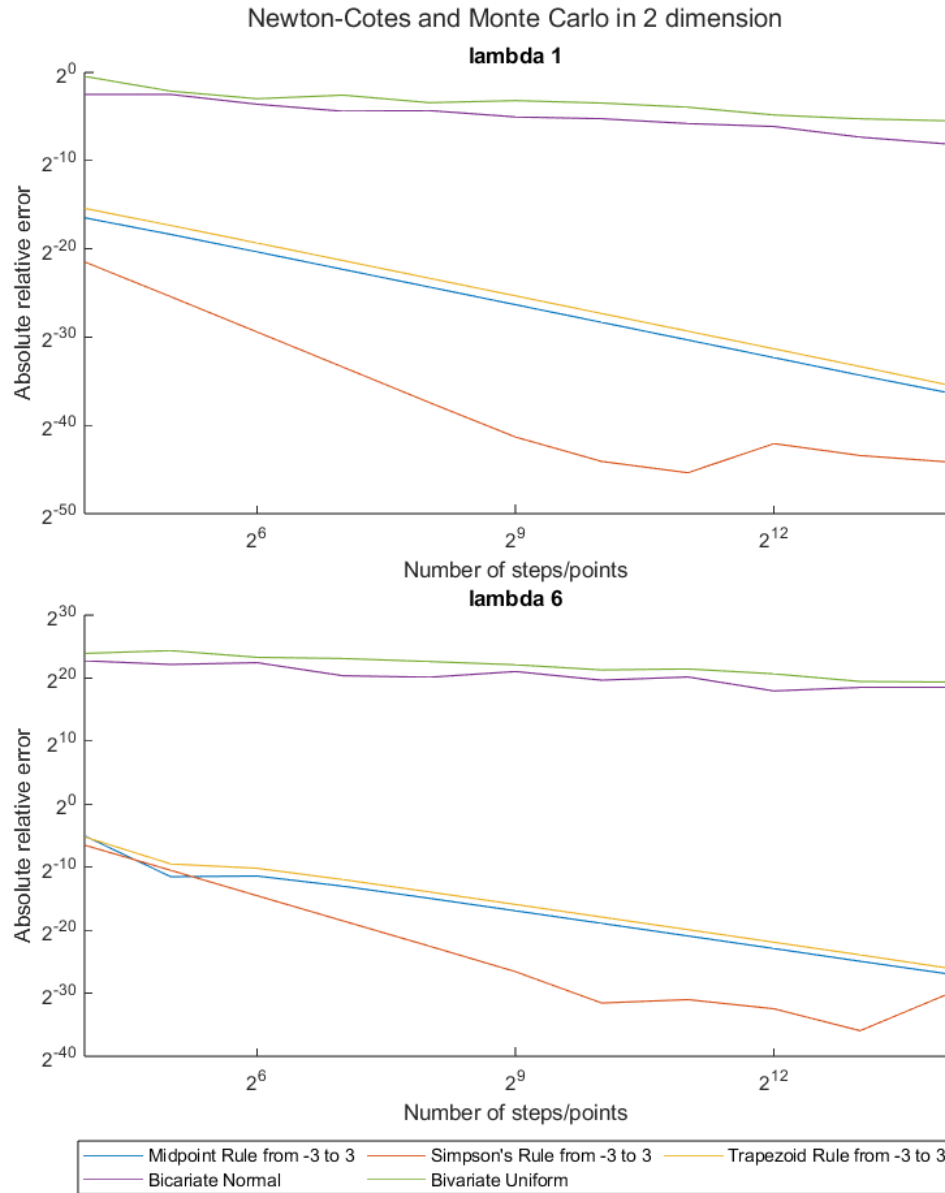


Figure 8: The upper plot is the log absolute relative error of the Newton-Cotes methods and Monte Carlo integration methods when $\lambda = 1$ and the lower plot is for $\lambda = 6$. Since 2D Newton-Cotes methods are evaluated using at most 2^{14} number of steps, the graphs compare the error with Monte Carlo for the same number of points. Made with `NCvsMCDataPlot.m`.

6 Conclusion

Throughout this project we have seen how the size of the error of the numerical approximation of the integral depends on the oscillatory behavior of the function, and how the computational complexity depends on the method and the dimension of the function. In particular, the computational complexity for Newton-Cotes methods grows exponentially as the dimension of the function increases. By comparison, the computational complexity of Monte Carlo integration methods grows linearly as the dimension of the function increases. We can therefore conclude that Monte Carlo integration is expected to be a better choice for numerically integrating higher dimensional functions, when only considering the computational complexity of the method.

However, we can also see that the error for Newton-Cotes methods is both generally lower than the error for Monte Carlo integration methods and decays at a faster rate as the number of points/steps increases. This can be seen in sections 5.3 and 5.6. Therefore, if the function that we wish to integrate is not a higher-dimensional function, Newton-Cotes methods offer more accurate approximations of the integral compared to Monte Carlo integration methods, without being too computationally intensive.

As for the oscillatory behavior of the function, for larger values of λ we can see that it increases the magnitude of the error for all the methods we used. This is because of the sign-problem. When λ is large, the integral additions from one step to the next are almost equal at certain points but with different signs. They therefore cancel each other out when added to the integral approximation. Since the value of the integral of our function gets smaller for larger values of λ , it also becomes harder to numerically approximate when machine precision affects the significant digits of our approximation.

There are ways to improve the numerical approximation of the integral or to optimize the computational complexity for the methods. For Newton-Cotes methods, a way to improve the computational complexity is to use adaptive step size. This does reduce the numerical accuracy of the method however. To reduce the error further, one can use Newton-Cotes methods with a higher order of error decay.

As for Monte Carlo integration, the error for importance sampling can be decreased by using a distribution with a higher correlation to the function we integrate. One way to get such a distribution is by using Markov Chain Monte Carlo methods like the Metropolis-Hastings algorithm.

7 Bibliography

- [1] R. A. Adams and C. Essex. *Calculus, A Complete Course*. Pearson, 2021.
- [2] D. Keffer. *Numerical Techniques for the Evaluation of Multi-Dimensional Integral Equations*. 1999.
- [3] G. Navas-Palencia. *Extending error function and related functions to complex arguments*. 2016.
- [4] A. C. Quillen. *PHY256 Notes on Importance sampling and the Metropolis-Hastings method*. 2021.
- [5] T. Sauer. *Numerical Analysis*. Pearson, 2014.
- [6] E. W. Weisstein. *Gaussian Integral - from Wolfram MathWorld - mathworld.wolfram.com*. <https://mathworld.wolfram.com/GaussianIntegral.html>. 2024.

8 Appendix

8.1 MATLAB code files and data

The files used to perform the numerical integrations and the generated data are in the appended zip file `BAMM501_code_file.zip`.